

Principes des lang. de progr. INE 213

François Pessaux
(Sur la base d'un cours de Michel Mauny)

ENSTA, édition 2025-2026

`prénom.nom@ensta.fr`



Langages de programmation

Introduction à OCaml

- 1 Préliminaires
 - Objectifs du cours
 - Organisation
- 2 Les langages de programmation
 - Techniques de mise en œuvre
- 3 OCaml
 - Opérations de base
 - Fonctions et filtrage
 - Modularité

Motivations

1/2

Au début était l'hexadécimal :

Langage machine MIPS R4000 (calcul du PCGD)

```
27bdfdd0 afbf0014 0c1002a8 0c1002a8 afa2001c 8fa4001c 00401825 10820008
0064082a 10200003 10000002 00832023 00641823 1483fffa 0064082a 0c1002b2
8fbf0014 27bd0020 03e00008 00001025
```

Puis vint le mnémonique :

Assembleur MIPS R4000

addiu	sp,sp,-32	b	C
sw	ra,20(sp)	subu	a0,a0,v1
jal	getint	subu	v1,v1,a0
jal	getint	C: bne	a0,v1,a
sw	v0,28(sp)	slt	at,v1,a0
lw	a0,28(sp)	D: jal	putint
move	v1,v0	lw	ra,20(sp)
beq	a0,v0,D	addiu	sp,sp,32
slt	at,v1,a0	jr	ra
A: beq	at,zero,B	move	v0,zero

Motivations

2/2

Et enfin les langages de haut niveau :

PGCD en C

```
#include <stdio.h>
#include <stdlib.h>

int pgcd(int i, int j) {
    if (i == j) return i;
    else if (i > j) return pgcd(i-j, j);
    else return pgcd(i, j-i);
}

int main(int argc, char **argv) {
    int n = atoi(argv[1]);
    int m = atoi(argv[2]);
    printf("%d\n", pgcd(n,m));
    exit(0);
}
```

En OCaml

```
let rec pgcd(i, j) =
  if i = j then i
  else if i > j then pgcd(i-j, j)
  else pgcd(i, j-i)
;;

let main () =
  let (a,b) =
    (int_of_string(Sys.argv.(1)),
     int_of_string(Sys.argv.(2)))
  in
  Printf.printf "%i\n" (pgcd(a,b));
  exit 0
;;

main ();;
```

Parenthèse : dans ce cours, on notera « $\leq$ », « $\neq$ », « $\rightarrow$ », ..., ce que dans les vrais programmes on écrit « $\leq$ », « $<$ », « $\rightarrow$ ».

Objectifs du cours

Développer une culture «langage»

- comprendre les langages de programmation *en général*
- avoir une idée de ce qui se cache derrière tel ou tel aspect de langage
 - comment ça marche ?
 - est-ce efficace ?
 - ce langage est-il adapté à cette tâche ?
- s'initier à OCaml.

Comment ?

- comprendre quelques bases de modèles d'exécution et de sémantique
- utiliser des outils de définition de langage
- programmer !

Ce cours n'est pas ...

Pas un cours de compilation

- génération de code, allocation de registres, optimisation, *etc.* sont hors du champ de ce cours (même si présents dans le poly).

Pas une introduction à tous les langages

- procédures/fonctions, gestion de la mémoire, objets
- **pas** de programmation logique ou par contraintes, (probablement) pas de mécanismes objets compliqués, pas de programmation concurrente, ni temps-réel, ni ...

Pas de la théorie pour la théorie

- ce dont on a besoin pour comprendre, utiliser, se familiariser, et prendre un peu de recul

Organisation

1/2

Séances

- 01 Tour d'horizon des techniques de mise en œuvre, premiers pas en OCaml
- 02 OCaml, la suite
- 03 Analyse lexicale (expressions rationnelles)
- 04 Analyse syntaxique (grammaires algébriques)
- 05 Sémantique dénotationnelle
- 06 Sémantique opérationnelle
- 07 Termes, filtrage, unification
- 08 Vérification et synthèse de types
- 09 Objets
- 10 Gestion automatique de la mémoire
- 11 Stratégies d'évaluation (le λ -calcul)
- 12 Introduction à l'interprétation abstraite
- 13 Récréation : quelques subtilités de langages

Organisation

2/2

Emploi du temps

- Présence obligatoire ; cours et TP commencent à l'heure !
- Cours : de 1h à 1h15, TP : max 2h ; pause : 15 min.

Extras

- Travail personnel : chapitres à lire, exercices à faire
- Aide, explications, discussions, après les cours

Documents :

- transparents, polycopié, documentation OCaml
- accessibles depuis la page web du cours

Contrôle des connaissances :

- les petits exercices de début de cours et votre projet

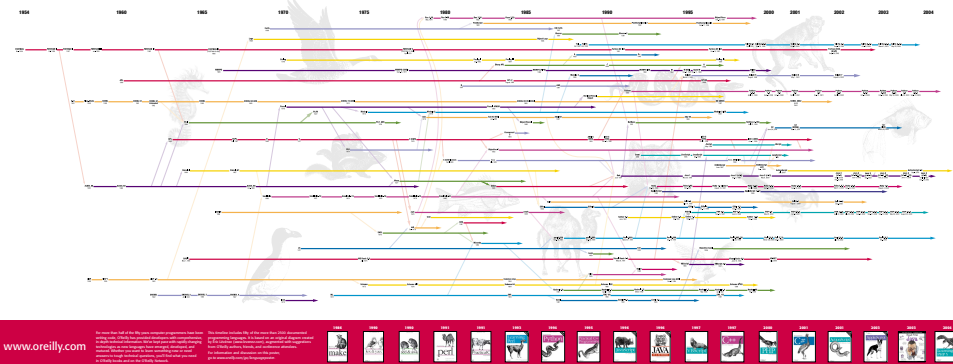
Page web du cours :

- Sur le Moodle, rubrique IN213

Le paysage

History of Programming Languages

O'REILLY



Les grandes lignes

- 1960 : le début
 - Fortran pour le calcul
 - Algol pour la science informatique
 - Lisp pour les structures de données dynamiques
- 1970 : des nouveautés...
 - Prolog
 - PL/1
 - Smalltalk
 - BASIC
- ... et des tendances
 - *Programming in the large* : Clu, Modula
 - Langages de scripts (sh, awk)
 - Programmation fonctionnelle (SASL)
- Les années 1980
 - Objets
 - Langages de scripts (awk, rexx, perl)
 - Programmation fonctionnelle (Miranda)
- Tendances
 - Java
 - Langages de scripts (JavaScript, Ruby, PHP)
 - Programmation fonctionnelle (Scheme, Caml, Haskell, SML)

Techniques de mise en œuvre

1/4

1. Pré-traiter

- cpp, m4, macros des langages Lisp
- agissent sur le texte source des programmes
- sans «comprendre» le programme (*i.e.* sans en analyser le sens)
- ni procéder à son exécution
- $\Rightarrow$ copier-coller textuel (parfois un peu intelligent)



Techniques de mise en œuvre

2/4

2. Compiler = traduire

- traduction d'un programme source en un programme «objet»
- donne du sens au programme : la traduction préserve le sens
- ne procède pas à l'exécution du programme (entrées non disponibles)



Attention

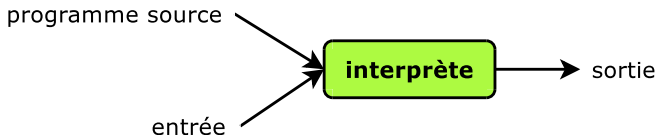
- «source» $\nrightarrow$ «langage haut niveau»
 - ▶ assembleur (*a priori* bas niveau) : traitements relevant de la compilation (optimisation, réordonnancement)
- «objet» $\nrightarrow$ «langage bas niveau»
 - ▶ compiler du Caml vers du C (*a priori* haut niveau)

Techniques de mise en œuvre

3/4

3. Interpréter = exécuter

- exécuter le programme **avec** ses données
- machine :
 - logicielle (virtuelle) $\Rightarrow$ nécessite un *runtime*
 - ou réelle $\Rightarrow$ CPU est directement l'interprète

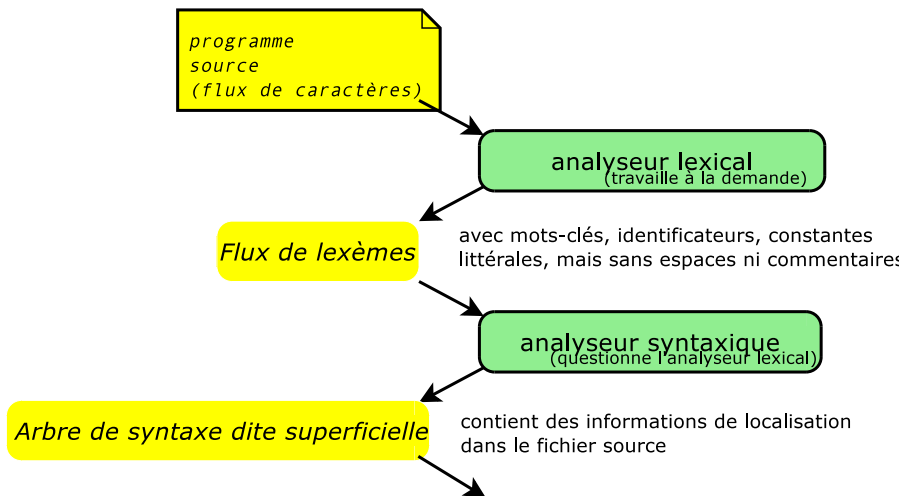


4/4



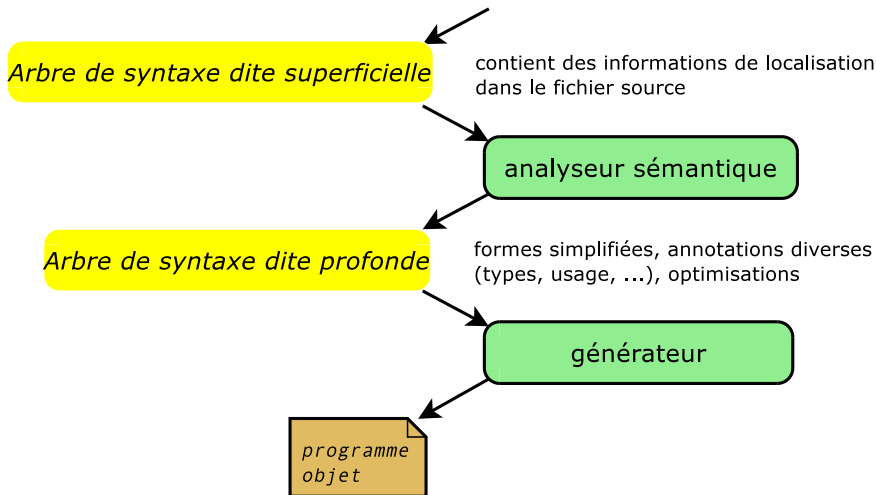
La compilation

1/2



La compilation

2/2



Le langage OCaml

Un peu d'histoire

- 1978 Robin Milner propose ML comme Méta-Langage pour décrire des stratégies dans un outil d'aide à la démonstration de théorèmes.
- 1981 Premières implémentations de ML.
- 1985 Développement de Caml à l'INRIA, de Standard ML à Edimbourg, puis à Bell Labs (New Jersey, USA).
- 1990 Développement de Caml-Light à l'INRIA.
- 1995 Ajout d'un compilateur natif et d'un système de modules.
- 1996 Ajout des objets.

Domaines d'utilisation

Langage algorithmique à usage général

Domaines de prédilection

- Calcul symbolique
- Prototypage rapide, langage de script
- Applications distribuées

Enseignement et recherche

- Laboratoires de recherche
- Classes préparatoires (OCaml a remplacé Caml-Light)
- Universités (Europe, USA, Japon, ...)

Industrie

- Jane Street, LexiFi, Tezos
- Facebook, BeSport, MyLife
- OCamlPro, Intel, Microsoft, IBM, SimCorp, Citrix (Xen), Google...
- The OCaml Software Foundation, Consortium Caml, OCamlLabs (Cambridge, UK)

OCaml : caractéristiques

OCaml est un langage

- fonctionnel : les fonctions sont des données (presque) comme les autres
- avec des traits impératifs
- fortement typé
- avec synthèse de types
- à gestion mémoire automatique
- avec des modules
- avec des objets
- compilé
- qui peut être interactif

Premiers pas

Éditer le source : `bienvenue.ml`

```
Printf.printf "Bienvenue !\n";;
```

Compiler et exécuter

```
$ ocamlc -o bv bienvenue.ml
```

```
$ ./bv
```

```
Bienvenue !
```

Premiers pas

En mode interactif

```
$ ocaml
      OCaml version ...
# Printf.printf "Bienvenue !\n";;
Bienvenue !
- : unit = ()
# let euro x = floor (100.0 *. x /. 6.55957) *. 0.01;;
val euro : float -> float = <fun>
# let baguette = 5.60;;
val baguette : float = 5.6
# euro baguette;;
- : float = 0.85
# exit 0;;
```

Intérêt

- Comme une calculatrice.
- Pour la mise au point des programmes
- Comme exécutant de scripts (`ocaml bienvenue.ml`).

Programmes OCaml

Un programme est une suite de phrases de la forme

let $x = e;;$	définition de x comme résultat du calcul de e
let $f\ x_1 \dots x_n = e;;$	définition de fonction
let rec $f\ x_1 \dots x_n = e;;$	définition de fonction récursive
let [rec] $f_1\ x_1^1 \dots x_{n_1}^1 = e_1$ and $\dots$ and $f_k\ x_n^k \dots x_{n_k}^k = e_k;;$	définition de fonctions mutuellement récursives
$e;;$	expression à évaluer
type $t_1 = \dots [\dots \text{and } t_n = \dots]$	définition de nouveaux types

où $e, e_1, \dots, e_k$ sont des **expressions**.

(* Commentaires (* imbriqués *) possibles $\Rightarrow$ analyse lexicale
des commentaires. Permet de commenter rapidement du code. *)

Expressions OCaml

Structures de données : création et «déconstruction»

23, 'z', true, "foobar", 3.14	(* Valeurs de types de base *)
if e ₁ then e ₂ else e ₃	(* Expression conditionnelle *)
"IN213" ['I'; 'N'; '2'; '1'; '3']	(* Données structurées allouées chaînes, tableaux *)
e.[i] e.(i)	(* Accès (chaîne, tableau) *)
(fact, (if ... then fact else succ), 2 * 3)	(* N-uplets *)
[] e ₁ ::e ₂ [0; 1; 3; 4]	(* Listes *)
match e with p ₁ -> e ₁ ... p _n -> e _n	(* Analyse de cas *)

Expressions OCaml

OCaml est un langage d'expressions

(* Fonction anonyme *)

fun $x_1 \dots x_n \rightarrow e$

(* Appels de fonction *)

$f\ e_1 \dots e_n$

$e_0\ e_1 \dots e_n$

(* Identificateurs *)

x (ou bien $M.x$ si x est défini dans M)

(e)

`fun x y -> 2 * x + y`

`fact (succ 5)`

`(if Random.bool () then fact else succ) 5`

`(fact 5)`

Expressions OCaml

OCaml est un langage d'expressions

Définitions locales

```
let x = e1 in e2
```

```
let x = e1 in
let y = e2 in
e3
```

```
let x = e1
and y = e2 in e3
```

```
let f x = e1 in e2
```

```
let rec f x y z = ... and g t = ... in ...
```

```
let [rec] f [x1 ... xn] = e1 [and ...] in e2
```

```
let a = 1 in
let b = a + 10 in
a * b
```

```
let a = 1
and b = 234 in
a * 1000 + b
```

```
let succ n = n + 1
```

```
let rec fact n =
  if n = 0 then 1 else
  n * fact (n - 1)
```

```
let rec even n =
  if n = 0 then true else odd (n - 1)
and odd m =
  if m = 0 then false else even (m - 1)
```

```
fact (let a = 1 in
      let b = a + 10 in
      a * b)
```

Expressions OCaml

Quelques expressions ressemblent à des instructions

for $i = e_1$ **to** e_2 **do** e_3 **done**
while e_1 **do** e_2 **done**

(* Boucles *)

$e_1; e_2$
 $(e_1; e_2)$
begin $e_1; e_2$ **end**

(* Évaluation en séquence *)

Remarques

- Fonctions : pas de notion de procédures
- Boucles **for** et **while** « ne fournissent pas de résultat » (significatif)
- Valeur **()** de type `unit` : « absence de valeur » (significative)

Opérations de base

type	valeurs	opérations
unit	()	
bool	true, false	&&, , not
char	'a', '\n', '\065'	Char.code, Char.chr
int	1, 2, 3	+, −, *, /, max_int
float	1.0, 2., 3.14	+. , −. , *. , /. , cos
string	"a\tb\010c\n"	^, s.[i]
tableaux	[0; 1; 2; 3]	t.(i), t.(i) <- v
n-uplets	(1, 2), (1, 2, 3, 4)	fst, snd

Infixe entre parenthèses devient **préfixe** : (+), (*), (mod)

Fonctions

Arguments sont passés sans parenthèses

```
let plus x y = x + y           : int -> int -> int
                                : int -> (int -> int)
let plus (x, y) = x + y       : (int * int) -> int
```

La seconde fonction a **un seul** argument qui est une **paire**.

Type

- $t_1 \rightarrow t_2$, type de fonction
 - d'argument de type t_1 ,
 - retournant une valeur de type t_2 .
- $t_1 \rightarrow t_2 \rightarrow t_3$ lu comme $: t_1 \rightarrow (t_2 \rightarrow t_3)$.

Fonctions : application

Application : simple juxtaposition

plus 1 3 *est lu comme* (plus 1) 3
let succ = plus 1 **in** succ 3

```
Array.create (2 * n) ("Hello " ^ "world!")
Array.create (f n) (g "abcd")
print_string "foobar\n"
print_int 314159
```

Application : partielle

- plus : int -> int -> int
 - ▶ i.e. plus : int -> (int -> int).
- plus 1 : int -> int (paramètre x mémorisé, figé à 1)
- Fonction à n arguments $\equiv$ fonction
 - à 1 argument
 - retournant fonction à $n - 1$ argument(s).

Fonctions récursives

Récursivité notée par le mot-clé **rec**

```
let rec fact n =  
  if n > 1 then n * fact (n - 1) else 1 ;;
```

Récursion mutuelle

```
let rec ping n =  
  if n > 0 then "ping" :: pong (n - 1) else [ ]  
and pong n =  
  if n > 0 then "pong" :: ping (n - 1) else [ ] ;;  
  
ping 4;;  
- : string list = ["ping"; "pong"; "ping"; "pong"]
```

N-uplets

N-uplets : hétérogènes, mais arité fixée par le type

- une paire $(1, 2) : \text{int} * \text{int}$
- un triplet $(1, 2, \text{true}) : \text{int} * \text{int} * \text{bool}$

sont **incompatibles** entre eux.

Projections : polymorphes pour une arité donnée

```
(fun (x, y, z) -> x) : ('a * 'b * 'c) -> 'a
```

Paires : cas particulier avec projections prédéfinies

```
fst : ('a * 'b) -> 'a
```

```
snd : ('a * 'b) -> 'b
```


Fonctions

Fonctions : valeurs comme les autres

```
let compose f g = fun x -> f (g x) ;;  
val compose : ('a -> 'b) -> ('c -> 'a) -> ('c -> 'b)  
  
let succ n = n + 1 ;;  
let plus_deux = compose succ succ ;;  
  
let symetrique f x y = f y x ;;  
val symetrique : ('a -> 'b -> 'c) -> 'b -> 'a -> 'c
```

Presque comme les autres

```
succ = succ ;;  
Exception: Invalid_argument "compare: functional value"
```

Fonctions

Fonctions anonymes

```
(fun x y -> x + y) 1 3 ;;  
- : int = 4
```

```
let ff = ( (fun x -> x+1), (fun x -> x+2) ) ;;  
val ff : (int -> int) * (int -> int) = (<fun>, <fun>)
```

Notations équivalentes

`let f(x) = e`

`let f x = e`

`let f = fun x -> e`

Exemples

Composer n fois une fonction avec elle-même

```
let rec puissance f n =  
  if n ≤ 0 then (fun x -> x)  
  else  
    let fn1 = puissance f (n - 1) in  
    fun x -> f (fn1 x) ;;  
val puissance : ('a -> 'a) -> int -> 'a -> 'a
```

Utilisations

```
let plus u v = puissance succ u v ;;  
let fois u v = puissance (plus u) v 0 ;;  
let puis u v = puissance (fois u) v 1 ;;  
puis 2 16 ;;  
- : int = 65536  
let marge = puissance (fun x -> " " ^ x) ;;  
val marge : int -> string -> string
```

Polymorphisme

Souvent **plusieurs types** possibles pour une même valeur :

```
let id x = x;;
```

```
id : int -> int
```

```
id : string -> string
```

```
id : ('a * 'a) -> ('a * 'a)
```

```
id : ('a -> 'b) -> ('a -> 'b)
```

```
id : 'a -> 'a
```

Type plus ou moins général

- il existe un « meilleur » type pour `id`, dont tous les autres sont **instances**
- $\forall 'a. 'a \rightarrow 'a$
- instances : `int -> int`, `('b -> 'b) -> ('b -> 'b)`

Polymorphisme

Le **let** est l'unique occasion de produire du polymorphisme

- `let id x = x in (id true, id 3)` oui
- `(fun id -> (id true, id 3)) (fun x -> x)` non

Le **let** « sans calculs » est l'occasion de produire du polymorphisme

```
let id =
  let z = calcul () in
  (fun x -> x) in
(id true, id 3) non
```

Limitation nécessaire

- évite de rendre polymorphe abusivement
`let v = ref [] in (v := [1] ; v := [true] ; 1 + head !v)`
- nommée « value restriction »
- critère de limitation subtil, hors du périmètre du cours

Polymorphisme

Types non complètement déterminés ('_a, '_b,...)

- Dans la boucle interactive :

```
let id x = x ;;
val id : 'a -> 'a = <fun>
let f = id id ;;
val f : '_a -> '_a = <fun>
```

- f n'est pas polymorphe : son type n'est pas complètement déterminé

```
f (1, 2) ;;
- : int * int = (1, 2)
f;;
- : int * int -> int * int = <fun>
```

Unions

Unions discriminées (type somme)

```
type carte = Petite of int | Valet | Dame | Roi ;;  
type vehicule =  
  Auto of marque * puissance | Velo of int * int ;;  
type ibtree =  
  | Leaf of int  
  | Node of (ibtree * ibtree) ;;
```

Syntaxe : les constructeurs de données commencent toujours par une majuscule.

Types paramétrés

```
type 'a réponse = Oui of 'a | Non ;;  
type 'a btree =  
  | Leaf of 'a  
  | Node of ('a btree * 'a btree) ;;
```

Unions

Types rékursifs

En OCaml

```
type 'a liste =  
| Vide  
| Cell of 'a * 'a liste
```

En C

```
struct cell {  
    int tete ;  
    struct cell *reste ;  
}
```

Listes

Ici

```
'a liste  
Vide  
Cell (e, reste)  
Cell(e1, Cell (e2, V))
```

Listes prédéfinies

```
'a list  
[ ]  
e :: reste  
[e1; e2]
```


Filtrage

Les valeurs sont souvent examinées par filtrage

```
let valeur c = match c with  
| Valet -> 11  
| Dame -> 12  
| Roi -> 13  
| Petite 1 -> 20  
| Petite x -> x
```

```
let valeur = function  
| Valet -> 11  
| ...
```

Opérer sur un type récursif de données

```
let rec longueur xs = match xs with  
| Vide -> 0  
| Cell (tete, reste) -> 1 + longueur reste
```

Filtrage : sémantique

Examen séquentiel des clauses

```
| p1 -> e1  
| ... -> ...  
| pn -> en
```

Filtrages incomplets

Si **aucune** clause ne filtre, levée d'**exception** (fatale).

Les motifs sont linéaires

Une variable **ne peut pas** être liée **plusieurs fois** dans le même motif : ~~(x, x)~~ -> ...

Filtrage : motifs complexes

Examen de motifs complexes

- Portion de motif non « intéressante » : utiliser `_` à la place d'une variable.

```
let f arg = match arg with
| ( [ Valet          ; Petite x], (Valet :: _ ) ) -> ...
| ( ((Roi | Petite _) :: _ )      , [ ]          ) -> ...
| ...
| _ -> ...
```

Filtrages incomplets, redondants

```
let rec longueur xs =  
  match xs with [] -> 0 ;;
```

Warning P: this pattern-matching is not exhaustive.

Here is an example of a value that is not matched:

```
_::__
```

```
val longueur : 'a list -> int = <fun>
```

```
let rec longueur xs = match xs with
```

```
| [] -> 0
```

```
| x1 :: xs -> 2
```

```
| x1 :: x2 :: xs -> 3 ;;
```

Warning U: this match case is unused.

```
val longueur : 'a list -> int = <fun>
```

Pause pour aujourd'hui

```
# let pause () =  
    Printf.printf "Assez de discours, pause puis pratique.\n"  
    ;;  
val pause : unit -> unit = <fun>  
# pause () ;;  
Assez de discours, pause puis pratique.  
- : unit = ()
```

Tableaux

Tableaux : homogènes et polymorphes

```
[| 0; 1; 3 |] : int array  
[| true; false |] : bool array
```

Projections : polymorphes

```
(fun x -> x.(0)) : 'a array -> 'a  
(fun t k x -> t.(k) <- x) : 'a array -> int -> 'a -> unit
```

Tableaux : toujours initialisés

```
Array.create : int -> 'a -> 'a array  
Array.create 3 "chaîne" : string array
```

Enregistrements (records, structures)

Nécessitent une déclaration

```
type monnaie = { nom : string; valeur : float } ;;  
  
let x = { nom = "euro"; valeur = 6.55957 } ;;  
  
x.valeur ;;
```

Peuvent être paramétrés

```
type 'a info = { nom : string; valeur : 'a } ;;  
  
let get_val x = x.valeur ;;  
val get_val : 'a info -> 'a = <fun>
```

Enregistrements

Champs mutables

```
type personne = { nom : string; mutable age : int }
```

```
let p = { nom = "Paul"; age = 23 } ;;
```

```
val p : personne = {nom="Paul"; age=23}
```

```
let anniversaire x = x.age <- x.age + 1 ;;
```

```
val anniversaire : personne -> unit = <fun>
```

```
p.nom <- "Clovis" ;;
```

```
Characters 0-17: the label nom is not mutable
```

⇒ Autorise la modification de la composante age (pas de l'autre).

Filtrage et enregistrements

Filtrage de champs d'enregistrements

```
let nom_de p = p.nom
```

```
let nom_de p = match p with { nom = n } -> n
```

```
let is_jean p = match p with  
| { nom = ("Jean" | "jean") } -> true  
| _ -> false
```

```
match e with  
| Oui ({ valeur = Petite x }, _) -> x  
| ...
```

Références

Passage d'arguments «par valeur»

- pas de `&x`, `*x` comme en C
- variables **non modifiables**
- seules valeurs modifiables : **champs d'enregistrements**

```
type 'a pointeur = { mutable contenu : 'a } ;;
```

En C :

```
int x = 0 ;  
x = x + 1 ;
```

En OCaml :

```
let x = { contenu = 0 } ;;  
x.contenu <- x.contenu + 1 ;;
```

Raccourci :

```
let x = ref 0  
x := !x + 1
```

En pratique, on se sert **relativement peu** des références.

La ligne de commande

Arguments passés à un exécutable

- Placés dans le tableau `Sys.argv` : string array
- Nom de commande : `Sys.argv.(0)`
- Nombre d'arguments : $(\text{Array.length Sys.argv}) - 1$

Exemple : La commande Unix `echo`

```
echo.ml
```

```
for i = 1 to (Array.length Sys.argv) - 1 do
  Printf.printf "%s " Sys.argv.(i)
done ;
Printf.printf "\n%!"; exit 0 ;;
```

Entrées/sorties

Canaux prédéfinis

```
stdin : in_channel  
stdout : out_channel  
stderr : out_channel
```

Ouverture de canaux

```
open_out : string -> out_channel  
open_in : string -> in_channel  
close_out : out_channel -> unit
```

Entrées/sorties

Lecture sur stdin

```
read_line : unit -> string  
read_int : unit -> int
```

Écriture sur la sortie standard

```
print_string : string -> unit  
print_int : int -> unit  
Printf.printf : ('a, out_channel, unit) format -> 'a
```

Écriture sur la sortie d'erreurs

```
prerr_string : string -> unit  
prerr_int : int -> unit  
Printf.eprintf : ('a, out_channel, unit) format -> 'a
```

Entrées/sorties

Lecture sur un canal

```
input_line : in_channel -> string  
input_char : in_channel -> char
```

Écriture sur un canal

```
output_string : out_channel -> string -> unit  
output_char : out_channel -> char -> unit
```

Printf

```
Printf.printf  
  "Bonjour %s, il %d est heures\n" nom age  
Printf.eprintf ...  
Printf.fprintf chan ...  
Printf.sprintf ...
```

Exceptions

Exemple

```
exception Absent ;;
```

```
let rec cherche_la_cle k xs = match xs with  
| (h, v) :: t ->  
    if h = k then v else cherche_la_cle k t  
| [ ] -> raise Absent ;;
```

```
let k =  
    try cherche_la_cle "Georges"  
    [ ("Louis", 14); ("Georges", 5) ]  
    with Absent -> 10 ;;
```

Exceptions

Syntaxe

- Définition (**phrase**) **exception** C [**of** τ] ;;
- Levée (expression) raise e
- Filtrage (expression) **try** e **with** $p_1 \rightarrow e_1 \dots \mid p_n \rightarrow e_n$

Analogie avec le filtrage de valeurs

try ... **with** ... vs. **match** ... **with** ...

Exceptions prédéfinies

Exception	Usage	Gravité
Invalid_argument of string	Accès en dehors des bornes	Forte
Failure of string	Tête de liste vide Renvoie le nom de la fonction	Moyenne
Not_found	Recherche infructueuse	Nulle
Match_failure of ...	Échec de filtrage	Fatale
End_of_file	Fin de fichier	Nulle

Exceptions

Sémantique

- le type `exn` est une union discriminée (type somme)
- levée d'exception $\Rightarrow$ **arrêt** de l'évaluation en cours et production d'une **valeur « exceptionnelle »** (du type `exn`)
- propagation** remontant les imbrications d'**appels de fonctions**
- traitement** d'exception possible **seul^t** si levée durant l'évaluation de `e` dans une construction

try `e` **with** | `p1 -> e1` | ... | `pn -> en`

- si `e` produit `v`, on renvoie `v`
- si `e` lève `xc`, filtrage de `xc` par | `p1 -> e1` | ... | `pn -> en`
 - si `pi` est le premier à filtrer `xc`, on renvoie la valeur de `ei`
 - sinon, on propage (*raise*) `xc` à la recherche d'un **try ... with ...** englobant

Exceptions

Usage

Définir de **nouvelles** exceptions plutôt que réutiliser les exceptions prédéfinies

On utilise les exceptions pour

- **reporter une erreur** et interrompre le calcul en cours ; par exemple `Match_failure`
- communiquer une valeur de retour **rare ou particulière** ; exemple : `Not_found` et `End_of_file`

Exemple

La commande Unix `cat` (version restreinte)

```
try
  while true do
    let li = read_line () in
    print_string li ;
    print_newline ()
  done
with End_of_file -> () ;;
```

Modules

Programmation modulaire

- découpage du programme en morceaux compilables indépendamment
pour rendre les gros programmes compilables
- donner de la structure au programme
pour rendre les gros programmes compréhensibles
- spécifier les liens (interfaces) entre les composantes
pour rendre les gros programmes maintenables
- identifier des sous-composantes indépendantes
pour rendre les composantes réutilisables

Modules de base

Structures : collections de **phrases** (sans ; ;)

```
struct
  p1
  ...
  pn
end
```

```
struct
  let x = ...
  module M = struct ... end
  module type S = sig ... end
  type ('a, 'b) t = ...
  exception Ok
end
```

Les p_i sont des

- **phrases** du langage (déclarations de types, de valeurs, d'exceptions)
- définitions de **sous-modules** **module** X = ...
- définitions de **types de module** **module type** S = ...

Modules

Création d'un module

```
module S = struct
  type t = int
  let x = 0
  let f x = x + 1
end
```

```
module T = struct
  type t = int
  let x = 0
  module U = struct
    let y = true
  end
  let f x = x + 1
end
```

Modules

Utilisation d'un module

référence aux composantes d'un module par *M.comp*

```
... (S.f S.x : S.t) ...
```

Directive open

- alternative à la notation pointée
- pour utilisation intensive d'un module particulier
- toutefois **déconseillée** car nuit à la lisibilité

```
open S
```

```
... (f x : t) ...
```

Modules

Un module peut être composante d'un autre module

```
module T = struct
  module R = struct
    let x = 0
  end
  let y = R.x + 1
end
```

La notation pointée s'étend en **chemin** :

```
module Q = struct
  let z = T.R.x
  open T.R      donne accès direct aux composantes de T.R
  ...
end
```


Modules

Signature : type de module = collection de « spécifications »

Signature

```
sig
  spec1
  ...
  specn
end
```

où chaque *spec_i* est de la forme :

val <i>x</i> : <i>s</i>	valeurs
type <i>t</i>	types abstraits
type <i>t</i> = <i>τ</i>	types manifestes
exception <i>E</i>	exceptions
module <i>X</i> : <i>M</i>	sous-modules
module type <i>S</i> [= <i>M</i>]	types de module

Nommage d'un type de module par **module type** *T* = ...

```
module type MYSIG = sig type t val x : t end
```

Modules

Le compilateur infère les signatures de modules

```
ocamlc -i fichier.ml
```

Module

```
module Exple = struct
  type t = int
  module R = struct
    let x = 0
  end
  let y = R.x + 1
end;;
```

Signature inférée

```
module Exple : sig
  type t = int
  module R : sig
    val x : int
  end
  val y : int
end
```

Modules

Masquage par signature : (*structure* : *signature*)

- vérifie que la structure **satisfait** la signature
- rend **inaccessibles** les parties de la structure qui **ne sont pas spécifiées** dans la signature ;
- produit un résultat qui peut être lié par **module** $M = \dots$

Sucre syntaxique :

module M : *signature* = *structure*

équivalent à

module $M = (\textit{structure} : \textit{signature})$

Modules

Restriction équivalentes

```
module S = (struct
  type t = int
  let x = 1
  let y = x + 1
end : sig
  type t
  val y : t
end) ;;
```

```
module type SIG = sig
  type t
  val y : t
end

module S : SIG = struct
  type t = int
  let x = 1
  let y = x + 1
end ;;
```

Utilisation

```
S.x ;;
```

Error: S.x is unbound

```
S.y + 1 ;;
```

Error: S.y is not of type int

Modules et compilation séparée

Une unité de compilation A se compose de deux fichiers :

- ① **implémentation** `a.ml` :
 - une suite de phrases
 - semblable à un intérieur de **struct** ... **end**
- ② **interface** `a.mli` (optionnel) :
 - une suite de spécifications
 - semblable à l'intérieur de **sig** ... **end**

Une autre unité de compilation B peut faire référence à A comme si c'était une structure, en utilisant la **notation pointée** `A.x` ou bien en faisant **open** A.

Compilation séparée

Fichiers sources : a.ml, a.mli, b.ml

Étapes de compilation :

```
ocamlc -c a.mli    compile l'interface de A  
                  crée a.cmi  
ocamlc -c a.ml     compile l'implémentation de A  
                  crée a.cmo  
ocamlc -c b.ml     compile l'implémentation de B  
                  crée b.cmo  
ocamlc -o monprog a.cmo b.cmo  
                  édition de liens finale  
                  ⇒ crée monprog
```

Modules paramétrés

Foncteur : fonction des modules dans les modules

Corps du foncteur **explicitement** paramétré par le paramètre de module *S*.

```
functor (S : signature) -> module
```

Exemple

```
module T = functor (S : SIG) -> struct  
  type u = S.t * S.t  
  let y = S.g (S.x)  
end
```

Application de foncteur

```
module T1 = T (S1)  
module T2 = T (S2)
```

T1 et T2 s'utilisent alors comme des structures ordinaires :

```
(T1.y : T2.u)
```

Par défaut, T1 et T2 **partagent entièrement** leur code.

Informations, documentations, tutoriels, communauté
d'utilisateurs :

- <http://ocaml.org/>

Principes des lang. de progr. INE 213

François Pessaux
(Sur la base d'un cours de Michel Mauny)

ENSTA, édition 2025-2026

`prénom.nom@ensta.fr`

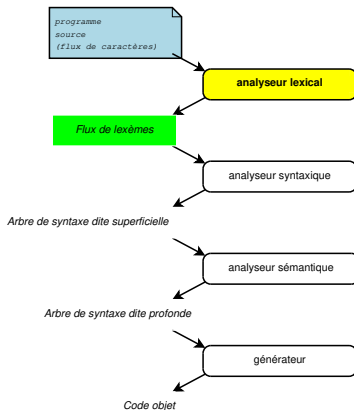


Analyse lexicale

- 1 Expressions rationnelles
- 2 Automates finis
- 3 OCamllex

Analyse lexicale : reconnaître les « mots »

Première phase d'un compilateur ou interprète



Analyse lexicale : reconnaître les « mots »

Première phase d'un compilateur ou interprète

- analyseur peut être écrit à la main
- le plus souvent produit automatiquement à partir d'un générateur
- spécifié à partir d'expressions rationnelles
- exécution : simulation d'un automate fini déterministe

Domaine bien étudié

- théorie bien comprise
- littérature abondante
- générateurs performants

Renouveau des automates

- vérification de conformité de XML vis-à-vis de DTD
- automates d'arbres pour traitement du XML

Lexèmes (les « mots »)

Exemple

```
while i <> j do (* commentaire *)
  if i > j then
    i := i-j
```

Résultat while ^{kwd}, i ^{ident}, ≠ ^{kwd}, j ^{ident}, do ^{kwd}, if ^{kwd}, ...

Représentation OCaml

```
type token = FOR | WHILE | DO | DONE | IF | THEN | ELSE | ...
| EQUAL | COLONEQUAL | PLUS | MINUS | STAR | SLASH | ...
| GREATER | SMALLER | SMALLERGREATER | ...
| IDENT of string | INT of int | FLOAT of float | ...
```

Résultat OCaml

```
WHILE; IDENT("i"); SMALLERGREATER; IDENT("j"); DO; IF; ...
```

Expressions rationnelles

Exemple 1 : entiers naturels

```
chiffre ::= '0' | '1' | '2' | ... | '7' | '8' | '9'  
naturel ::= chiffre chiffre*
```

Exemple 2 : identificateurs OCaml

```
majuscule ::= ['A'-'Z']           minuscule ::= ['a'-'z']  
lettre    ::= majuscule | minuscule  
constr    ::= majuscule (lettre | chiffre | '_')*  
ident     ::= minuscule (lettre | chiffre | '_')*
```

Expressions rationnelles

Définition

Expressions rationnelles

Soit Σ *alphabet*. Une expression rationnelle e est :

- le **mot vide** noté ϵ
- $a \in \Sigma$
- la **concaténation** de deux e.r. $e_1 e_2$
- une **alternative** $e_1 | e_2$
- une **répétition** e^*

Notations

$e_1 e_2 | e_3 e_4$ sera lu comme $(e_1 e_2) | (e_3 e_4)$

$e_1 e_2^* | e_3$ sera lu comme $(e_1 (e_2^*)) | e_3$

Expressions rationnelles

Formes dites dérivées (notations définissables)

- $e? \equiv e|\epsilon$
- $e^+ \equiv ee^*$
- $[abc] \equiv (a|b|c)$
- $[a_1-a_2] \equiv$ l'un des éléments de Σ compris entre a_1 et a_2 (Σ totalement ordonné)
- $[\wedge abc] \equiv$ l'un des éléments du complémentaire de $\{a, b, c\}$ dans Σ

Petites différences syntaxiques selon les outils (Emacs, Perl, Bash. . .)

Une expression représente un langage

Définition

Langage représenté par une expression rationnelle

$\mathcal{L}(e)$ est défini par :

- $\mathcal{L}(\epsilon) = \{\epsilon\}$
- $\mathcal{L}(a) = \{a\}$, où $a \in \Sigma$
- $\mathcal{L}(e_1 e_2) = \{m_1 m_2 \text{ avec } m_1 \in \mathcal{L}(e_1) \text{ et } m_2 \in \mathcal{L}(e_2)\}$
- $\mathcal{L}(e_1 | e_2) = \mathcal{L}(e_1) \cup \mathcal{L}(e_2)$
- $\begin{aligned} \mathcal{L}(e^*) &= \{\epsilon\} \cup \mathcal{L}(e) \cup \mathcal{L}(ee) \cup \dots \\ &= \bigcup_{i=0}^{\infty} \mathcal{L}(e^i) \end{aligned}$

Usage

Utilisations

- analyse lexicale (compilation de langages de programmation)
- éditeurs de texte
- motifs à rechercher dans fichiers / données (Unix, Web)
- DTD (qui indique la forme de documents XML)
- bibliothèques de langages (awk, Perl, Python, OCaml, PHP, ...) pour traitement de fichiers texte
- ...

Automates finis

Définition

Automate fini déterministe (DFA)

C'est la donnée de :

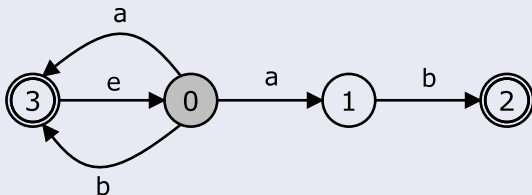
- un **alphabet** fini Σ ;
- un ensemble fini d'**états** Q ;
- une **fonction de transition** $\delta : Q \times \Sigma \rightarrow Q$;
- un **état initial** p_0 ;
- un ensemble d'**états finaux** $F \subset Q$.

Automate fini non déterministe (NFA)

- $\delta : Q \times (\Sigma \cup \{\epsilon\}) \rightarrow \mathcal{P}(Q)$
- $\mathcal{P}(Q) \Rightarrow$ **plusieurs** états suivants possibles.

Automates finis non déterministes

Automate fini non déterministe



Comment interpréter un automate fini non déterministe ?

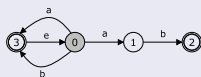
- maintenir **plusieurs** états en même temps

Liens DFAs/NFAs

- équivalence entre DFAs et NFAs
- un algorithme de détermination
- $\#$ états du DFA **potentiellement exponentiel** $\#$ états du NFA

Déterminisation : de NFA vers DFA

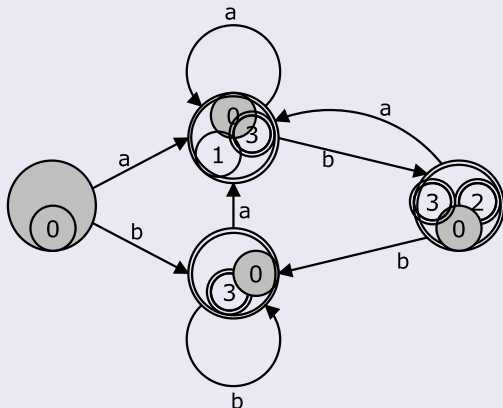
À déterminer



Mieux faire ?

- oui : minimiser (regrouper états équivalents)
- ou construire directement un automate déterministe (eventuellement minimal)

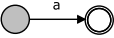
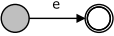
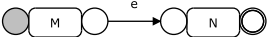
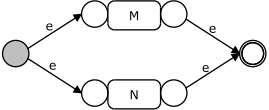
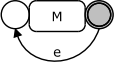
Résultat



Voir littérature / photocopié

Compilation d'expressions rationnelles

Traduire les expressions rationnelles en automates non-déterministes

Si l'expression est	on produit
$a \in \Sigma$	
ϵ	
MN	
$M N$	
M^*	

Génération d'analyseurs

Comment produire un analyseur à partir d'une spécification

- traduction d'expressions rationnelles en automates finis non-déterministes
- détermination
- minimisation

Génération directe de l'automate fini déterministe minimal

OCamllex

Générateur d'analyseurs lexicaux ocamllex

- monlexer.mll produit monlexer.ml
- tables ou fonctions récursives (option)

Structure

```
{ header }  
  
let ident = regexp ...  
  
rule next1 arg1... argn =  
  parse regexp { action }  
    | ...  
    | regexp { action }  
and nextk arg1... argm =  
  parse ...  
and ...  
  
{ trailer }
```

→ Prélude recopié tel quel dans le fichier produit

→ Définitions d'expressions rationnelles.

`nexti : ... → Lexing.lexbuf →  $\tau$`
`lexbuf : Lexing.lexbuf` est un paramètre implicite

Les analyseurs sont mutuellement récursifs.

Lorsque plusieurs regexp reconnaissent le même préfixe de l'entrée, c'est le cas qui reconnaît *l'entrée la plus longue* qui est choisi

→ Postlude recopié tel quel dans le fichier produit

Utilisation

Fonctions générées

- analyseurs :

$next_1 : [\text{types des paramètres} \rightarrow] \text{Lexing.lexbuf} \rightarrow \tau_1$

$next_2 : [\text{types des paramètres} \rightarrow] \text{Lexing.lexbuf} \rightarrow \tau_2$

...

- note : souvent pas de paramètres aux règles
- création de *buffers* :

$\text{Lexing.from_channel} : \text{in_channel} \rightarrow \text{lexbuf}$

$\text{Lexing.from_string} : \text{string} \rightarrow \text{lexbuf}$

Exemple : source

naturel.mll

```
{ exception Erreur }

let chiffre_non_nul = ['1'-'9']

rule nat = parse
  | chiffre_non_nul ('0' | chiffre_non_nul)*
    { int_of_string (Lexing.lexeme lexbuf) }
  | [' ' '\n' '\t']
    { nat lexbuf }
  | _ { raise Erreur (* Le cas _ doit venir en dernier. *) }

{ }
```

Note : expression rationnelles prédéfinie pour fin de fichier : eof

Exemple : source

naturel.mll

```
{ exception Erreur }

let chiffre_non_nul = ['1'-'9']

rule nat = parse
  | chiffre_non_nul ('0' | chiffre_non_nul)* as str
    { int_of_string str }
  | [ ' ' '\n' '\t' ]
    { nat lexbuf }
  | _ { raise Erreur (* Le cas _ doit venir en dernier. *) }

{ }
```

Compilation vers des tables

```
$ ocamllex naturel.mll
```

```
4 states, 267 transitions, table size 1092 bytes
```

```
$ cat naturel.ml
```

```
exception Erreur
```

```
let __ocaml_lex_tables = { ... les tables de l'automate ... }
```

```
let rec nat lexbuf = __ocaml_lex_nat_rec lexbuf 0
```

```
and __ocaml_lex_nat_rec lexbuf __ocaml_lex_state =
```

```
  match Lexing.engine __ocaml_lex_tables __ocaml_lex_state lexbuf with
```

```
  | 0 →
```

```
    let str = Lexing.sub_lexeme lexbuf
```

```
        lexbuf.Lexing.lex_start_pos lexbuf.Lexing.lex_curr_pos in
```

```
    ( int_of_string str )
```

```
  | 1 → ( nat lexbuf )
```

```
  | 2 → ( raise Erreur )
```

```
  | __ocaml_lex_state →
```

```
    lexbuf.Lexing.refill_buff lexbuf;
```

```
    __ocaml_lex_nat_rec lexbuf __ocaml_lex_state
```

```
::
```

Exemple : utilisation

```
$ ocaml
      OCaml version ...

# #load "naturel.cmo" ;;
# Naturel.nat ;;
- : Lexing.lexbuf → int

# Naturel.nat (Lexing.from_string "1234abcd") ;;
- : int = 1234

# let buf = Lexing.from_string "1234abcd" ;;
val buf : Lexing.lexbuf = { ... }

# Naturel.nat buf ;;
- : int = 1234

# Naturel.nat buf ;;
Exception: Naturel.Erreur.
```

Conclusion

- Analyse **lexicale** $\approx$ motifs à rechercher.
- Expressions **rationnelles** = **automates** finis.
- **Génération automatique** de code performant à partir de spécifications utilisant des expressions rationnelles.

Principes des lang. de progr. INE 213

François Pessaux
(Sur la base d'un cours de Michel Mauny)

ENSTA, édition 2025-2026

`prénom.nom@ensta.fr`



Analyse syntaxique

1 Grammaires

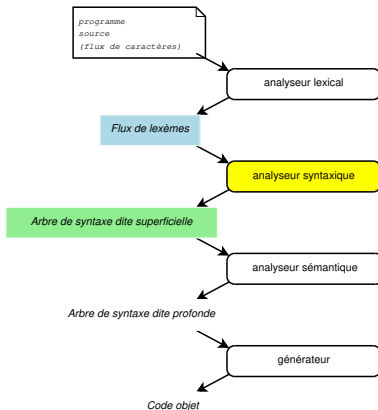
2 Analyse

- Analyse descendante
- Analyse ascendante

3 OCamlyacc

Analyse syntaxique

Seconde phase d'un compilateur ou interprète



Analyse syntaxique

Seconde phase d'un compilateur ou interprète

- analyseur peut être écrit à la main
- le plus souvent produit **automatiquement** à partir d'un générateur
- spécifié à partir d'une **grammaire**

Domaine bien étudié

- théorie bien comprise
- littérature abondante
- générateurs **performants**

L'analyseur syntaxique

- **refuse** des programmes n'appartenant pas au **langage** (auxquels on ne saurait pas donner de sens)
- $\approx$ « refuse les phrases grammaticalement incorrectes »
- construit une **représentation structurée** des programmes acceptés

Grammaires

On définit un langage à l'aide d'une **grammaire** :

$\text{digit} \rightarrow '0' \mid '1' \mid '2' \mid '3' \mid '4' \mid '5' \mid '6' \mid '7' \mid '8' \mid '9'$

$\text{nat} \rightarrow \text{chiffre} \mid \text{chiffre nat}$

$\text{expr} \rightarrow \text{nat} \mid \text{expr} '+' \text{expr}$

Une grammaire G est un quadruplet (N, T, P, S) où :

- N est un ensemble fini de symboles dits **non terminaux**
- T est un ensemble de symboles dits **terminaux**
- P est l'ensemble des **productions** : c'est un sous-ensemble de

$$(N \cup T)^* N (N \cup T)^* \times (N \cup T)^*$$

On note $\alpha \rightarrow \beta$ un élément (α, β) de P

- S est un symbole particulier de N et est appelé le **symbole de départ**.

Grammaires

Notation :

on regroupe les règles $\alpha \rightarrow \beta_i$ ayant le même membre gauche α , en

$$\alpha \rightarrow \beta_1 | \beta_2 | \dots | \beta_n$$

Notation BNF (Backus-Naur Form) : c'est le nom qu'on donne à ces notations de grammaire. Utilisée pour la définition du langage Algol (≈ 1960)

Notation EBNF (Extended BNF) :

- BNF
- + crochets $[\dots]$ pour des éléments optionnels
- + accolades $\{\dots\}$ pour l'étoile et $\{\dots\}^+$

Hiérarchie de Chomsky

Hiérarchie de Chomsky . Une grammaire est dite :

- de type 3 ou **rationnelle** si chaque production est ou bien de la forme $A \rightarrow wB$ ou $A \rightarrow w$ où A et B sont des non terminaux et w est un terminal, ou alors de la forme $A \rightarrow \epsilon$;
- de type 2 ou **algébrique** ou **non contextuelle** si chaque production est de la forme $A \rightarrow \alpha$ où A est un non terminal et α est une séquence de terminaux ou non terminaux ;
- de type 1 ou **contextuelle** si chaque production est de la forme $\alpha A \beta \rightarrow \alpha \gamma \beta$ où α , β et γ sont des séquences de terminaux ou non terminaux et $\gamma \neq \epsilon$;
- de type 0 ou **générale** si chaque production est de la forme $\alpha \rightarrow \beta$, sans autre contrainte.

Exemples

Rationnelle : a^*bc^*

$$\begin{array}{ll} S \rightarrow aS & S \rightarrow bC \\ C \rightarrow \epsilon & C \rightarrow cC \end{array}$$

Algébrique (non contextuelle) :
 a^ib^i avec $i > 0$

$$\begin{array}{l} S \rightarrow aSb \\ S \rightarrow ab \end{array}$$

Contextuelle : $a^ib^jc^i$ avec $i > 0$

$$\begin{array}{ll} S \rightarrow aSBC & S \rightarrow aBC \\ Ca \rightarrow aC & Ba \rightarrow aB \\ CB \rightarrow BC & aB \rightarrow ab \\ bB \rightarrow bb & bC \rightarrow bc \\ cC \rightarrow cc & \end{array}$$

Grammaires algébriques

Productions de la forme

$$A \rightarrow \alpha$$

Langage $\mathcal{L}(G)$ engendré par G : l'ensemble des **suites finies** w de **terminaux** que l'on peut dériver de S par la méthode suivante, où α est ce qui a été dérivé jusqu'alors (initialement, $\alpha = S$) :

- 1 si α est une suite w composée uniquement de **terminaux**, alors $w \in \mathcal{L}(G)$;
- 2 sinon $\alpha = \beta A \gamma$ où A est un **non terminal** : considérons une production $A \rightarrow \delta$ et remplaçons α par $\beta \delta \gamma$, et recommencer en 1.

Exemple

Grammaire des expressions arithmétiques

$$S \rightarrow E$$

$$E \rightarrow E + E$$

$$E \rightarrow E - E$$

$$E \rightarrow E * E$$

$$E \rightarrow E / E$$

$$E \rightarrow (E)$$

$$E \rightarrow \text{INT}$$

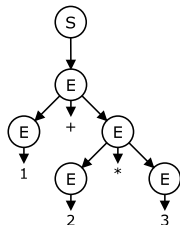
Terminaux : constantes entières, opérateurs et parenthèses.

Non-terminaux : E et S , et **symbole de départ** : S .

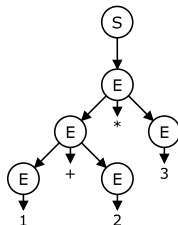
Dérivations, arbres d'analyse

Plusieurs dérivations possibles pour $1 + 2 * 3$

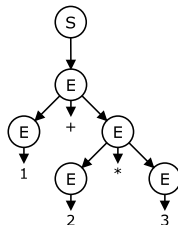
$S \Rightarrow E$
 $\Rightarrow E + E$
 $\Rightarrow INT_1 + E$
 $\Rightarrow INT_1 + E * E$
 $\Rightarrow INT_1 + INT_2 * E$
 $\Rightarrow INT_1 + INT_2 * E$
 $\Rightarrow INT_1 + INT_2 * INT_3$



$S \Rightarrow E$
 $\Rightarrow E * E$
 $\Rightarrow E + E * E$
 $\Rightarrow INT_1 + E * E$
 $\Rightarrow INT_1 + INT_2 * E$
 $\Rightarrow INT_1 + INT_2 * E$
 $\Rightarrow INT_1 + INT_2 * INT_3$

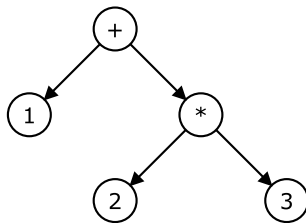
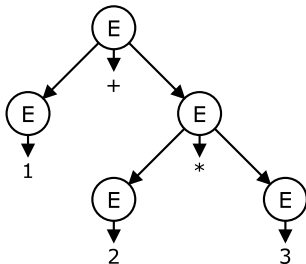


$S \Rightarrow E$
 $\Rightarrow E + E$
 $\Rightarrow E + E * E$
 $\Rightarrow E + E * INT_3$
 $\Rightarrow E + INT_2 * INT_3$
 $\Rightarrow INT_1 + INT_2 * INT_3$



Arbres de syntaxe

Arbre de syntaxe : étiqueter les nœuds par les lexèmes.



Analyse descendante

Interpréter les **non-terminaux** comme des **fonctions** :

`parse_E` : flux d'entrée $\rightarrow$ oui/non (ou `Some (arbre)/None`)

$$\begin{array}{lcl} E & \rightarrow & E + E \\ E & \rightarrow & E - E \\ \dots & & \\ E & \rightarrow & (E) \\ E & \rightarrow & \text{INT} \end{array}$$

```
parse_E () = parse_E () ;  
  regarder lexème courant  
  si '+' : demander nouveau lexème ; parse_E ()  
  si '-' : demander nouveau lexème ; parse_E ()  
  ...
```

Problème :

- récursion **gauche** : `parse_E` appelée **indéfiniment**

$\Rightarrow$ Transformer la grammaire : dé-récursiver à gauche

Transformation de grammaires

Éliminer la récursion gauche :

$$\begin{aligned} A &\rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_n \\ &\Rightarrow \\ A &\rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_n A' \\ A' &\rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_m A' \mid \epsilon \end{aligned}$$

$$S \rightarrow P + S$$

$$S \rightarrow P - S$$

$$S \rightarrow P$$

$$P \rightarrow F * P$$

$$P \rightarrow F / P$$

$$P \rightarrow F$$

$$F \rightarrow (S)$$

$$F \rightarrow \text{INT}$$

Analyse ascendante

Principe

- empiler les lexèmes (*shift*)
- réduire (*reduce*) une production (remplacer le segment terminal α de la pile par A si $A \rightarrow \alpha$)
- succès si on arrive à réduire S

Implémentation

- ne s'écrit jamais à la main
- générateurs efficaces (Yacc, Bison, ...) d'analyseurs à partir de grammaires :
 - 1 analyse de la grammaire, messages d'avertissement éventuels
 - 2 génération d'un automate à pile (*pushdown automaton*)
- LR (*Left-to-right scan, Rightmost derivation*)

Généralités

Avantages

- lisibilité,
- maintenabilité, évolutivité,
- efficacité

Inconvénients

- mise au point quelquefois difficile (gestion des conflits)

OCamlyacc

Générateur d'analyseurs ascendants pour OCaml

- similaire à Yacc (pour C, mais aussi Java, etc)
- analyseur LALR(1) : LR(1) + compaction de tables
- grammaire + actions dans un fichier foobar.mly
- résultat dans foobar.mli et foobar.ml

OCamlyacc

Structure de parser.mly

```
%{  
  prélude                                (* Commentaires OCaml *)  
%}
```

```
  déclarations                            /* Commentaires à la C */
```

```
%%
```

```
  règles                                  /* Commentaires à la C */
```

```
%%
```

```
  postlude                                (* Commentaires OCaml *)
```

Structure de parser.mly

Prélude : code OCaml, recopié dans le résultat

Déclarations :

```
%token <int> INT                /* Déclaration des lexèmes */
%token PLUS MINUS TIMES DIV
%token LPAREN RPAREN
%token EOL

%left PLUS MINUS                /* Précédence faible */
%left TIMES DIV                 /* Précédence moyenne */
%nonassoc UMINUS                /* Précédence forte */

%start main                     /* Le point d'entrée... */
%type <int> main                /* ... et son type */
```

Précédence : priorité par rapport aux autres opérateurs

Associativité : parenthésage entre plusieurs occurrences du même opérateur binaire

Structure de parser.mly

Règles : productions de grammaire + **actions**

- productions : séquences de terminaux et non-terminaux
- actions : **expressions OCaml** entre { accolades }

```
main: expr EOL                { $1 }  
;  
expr:  
    INT                        { $1 }  
  | LPAREN expr RPAREN        { $2 }  
  | expr PLUS expr             { $1 + $3 }  
  | expr MINUS expr            { $1 - $3 }  
  | expr TIMES expr             { $1 * $3 }  
  | expr DIV expr               { $1 / $3 }  
  | MINUS expr %prec UMINUS { - $2 }  
;
```

Dans les actions, les résultats de non-terminaux et les valeurs des lexèmes sont représentés par des \$1, \$2, ...

OCamlyacc : interface de l'analyseur

```
$ ocamlyacc parser.mly
```

produit le fichier d'**interface** `parser.mli` :

```
type token =  
  INT of (int) | PLUS | MINUS | TIMES  
  | DIV | LPAREN | RPAREN | EOL  
  
val main :  
  (Lexing.lexbuf → token) → Lexing.lexbuf → int
```

et son **implémentation** `parser.ml`.

OCamlyacc : programme principal

Usage :

```
let _ =  
  try  
    (* Créer un channel de lexing. *)  
    let lexbuf = Lexing.from_channel stdin in  
    while true do  
      (* Appeler le parser en lui donnant le lexer et le channel. *)  
      let result = Parser.main Lexer.token lexbuf in  
      Printf.printf "%d\n%!" result  
    done  
  with Lexer.Eof → exit 0
```

⇒ analyseur syntaxique est maître

► il appelle l'analyseur lexical selon le besoin

Arbres de syntaxe abstraite

Exemple précédent : éval directe d'une expression arithmétique

Traitement au vol pas toujours possible

⇒ besoin de créer un **arbre de syntaxe abstraite** (AST)

Type structuré OCaml (généralement récursif)

```
type aexpr =  
  | Intconst of int  
  | Addition of aexpr * aexpr  
  | Subtraction of aexpr * aexpr  
  | Multiplication of aexpr * aexpr  
  | Division of aexpr * aexpr  
  | Uminus of aexpr
```

Production d'arbres de syntaxe

expr:

INT	{ Intconst \$1 }
LPAREN expr RPAREN	{ \$2 }
expr PLUS expr	{ Addition (\$1, \$3) }
expr MINUS expr	{ Subtraction (\$1, \$3) }
expr TIMES expr	{ Multiplication (\$1, \$3) }
expr DIV expr	{ Division (\$1, \$3) }
MINUS expr %prec UMINUS	{ Uminus (\$2) }

;

Conclusion

Hiérarchie des langages/grammaires

- 3 : rationnel ($\approx$ lexique des langages de programmation)
- 2 : algébrique ($\approx$ syntaxe langages de programmation)
- 1 : contextuel
- 0 : général

Syntaxe des langages

- grammaires algébriques
- analyse descendante, programmation directe
- analyse ascendante : Yacc

Principes des lang. de progr. INE 213

François Pessaux
(Sur la base d'un cours de Michel Mauny)

ENSTA, édition 2025-2026

`prénom.nom@ensta.fr`



Sémantique dénotationnelle

- 1 Sémantique dénotationnelle directe
 - Domaines
 - Fonctions sémantiques
 - Instructions et aspects impératifs
- 2 Sémantique dénotationnelle à continuations

Sémantique

Syntaxe vs sémantique

- notation
- sens, dénotation

Sens d'un programme : peut décrire

- ce qu'il calcule
- l'objet mathématique qu'il représente
- comment il le calcule

Sur quoi faire reposer le sens d'un programme ?

- une description en langue naturelle
- un interprète : implémentation de l'interprète
- un compilateur (traducteur) : langage cible, implémentation du compilateur
- un formalisme mathématique

Sémantique

Le sens d'un langage doit (devrait) être

complet, cohérent, précis (non ambigu), concis, compréhensible

Sémantique formelle

- permettre la **standardisation** (non ambigu, précis, concis)
- **référence** pour **implémenteurs**, contre les implémentations divergentes du même langage
- implémentation automatique : sémantique exécutable
- **référence** pour **programmeurs** : savoir « ce que ça fait » (sans le tester)
- **preuves de correction** de programmes : raisonnement mathématique **impossible** sans sémantique formelle
- meilleure compréhension de la **conception des langages** (distinguer ce qui est compliqué de ce qui ne l'est pas)

Sémantique(s)

Sémantique dénotationnelle [C. Strachey, D. Scott, ~1960]

- définir l'objet **mathématique** représenté par un programme
- syntaxe $\rightarrow$ domaines mathématiques

Sémantique axiomatique [T. Hoare, 1969]

- définir l'**action** d'une construction par les propriétés de la **mémoire** avant et après son exécution
- $\{p\}S\{q\}$, préconditions, postconditions

Sémantique opérationnelle [G. Plotkin, 1981]

- décrire le mécanisme d'**évaluation** / **exécution** des programmes
 - **plus précise** que la sémantique dénotationnelle
- $\Rightarrow$ *présentation de la S.O. dans un des cours suivants*

PCF

Catégories

$c \in \text{Const}$ Constantes, incluant les fonctions primitives
 $x \in \text{Id}$ Identificateurs
 $e \in \text{Exp}$ Expressions

Expressions

$e ::= c \mid x$
 $\mid e_1 + e_2 \mid e_1 = e_2 \mid \text{if } e_1 \text{ then } e_2 \text{ else } e_3$
 $\mid \text{let } x = e_1 \text{ in } e_2 \mid \text{let rec } x = e_1 \text{ in } e_2$
 $\mid (\text{fun } x \rightarrow e) \mid e_1 e_2$

Exemple

**let rec fact = fun n → if n ≤ 0 then 1 else n * fact (n - 1) in
fact (5)**

Domaines

Domaines pour les valeurs, etc. : ordres partiels complets (CPO)

- éléments partiellement indéfinis, voire totalement indéfini ($\perp$)
 - ▶ représenter programmes qui ne terminent pas
- relation d'ordre partiel « moins défini que »
- toute chaîne¹ **strictement croissante** a une borne sup.

1. Partie totalement ordonnée

Domaines pour PCF

Domaine D

- INT : domaine des entiers
- BOOL = $\{ \text{true}, \text{false} \}$
- FUN = $D \rightarrow D$: fonctions sur D

$\Rightarrow D = \text{INT} + \text{BOOL} + \text{FUN}$: domaine des valeurs

Environnements : fonctions associant des valeurs de D à des identificateurs de Id

- ENV = $\text{Id} \rightarrow D$

Fonctions sémantiques

Sémantique des expressions

$$\mathcal{E} : \text{Exp} \rightarrow \text{ENV} \rightarrow D$$

Définition

$$\mathcal{E}[\![c]\!]\rho = d_c \text{ pour chaque } c \in \text{Const}$$

$$\mathcal{E}[\![x]\!]\rho = \rho[x], \text{ pour } x \in \text{Id}$$

$$\mathcal{E}[\![e_1 + e_2]\!]\rho = \mathcal{E}[\![e_1]\!]\rho +_D \mathcal{E}[\![e_2]\!]\rho$$

$$\mathcal{E}[\![e_1 \ e_2]\!]\rho = (\mathcal{E}[\![e_1]\!]\rho) (\mathcal{E}[\![e_2]\!]\rho)$$

$$\begin{aligned} \mathcal{E}[\![e_1 = e_2]\!]\rho &= \text{true} && \text{si } \mathcal{E}[\![e_1]\!]\rho = \mathcal{E}[\![e_2]\!]\rho \\ &= \text{false} && \text{dans le cas contraire} \end{aligned}$$

$$\begin{aligned} \mathcal{E}[\![\text{if } e_1 \text{ then } e_2 \text{ else } e_3]\!]\rho = \\ \text{si } \mathcal{E}[\![e_1]\!]\rho = \text{true} \text{ alors } \mathcal{E}[\![e_2]\!]\rho \text{ sinon } \mathcal{E}[\![e_3]\!]\rho \end{aligned}$$

Fonctions sémantiques : décls., fonctions

Sémantique des expressions (suite)

$$\mathcal{E}[\text{let } x = e_1 \text{ in } e_2]\rho = \mathcal{E}[e_2](\rho \oplus [x \mapsto \mathcal{E}[e_1]\rho])$$

$$\mathcal{E}[\text{let rec } f = e_1 \text{ in } e_2]\rho = \mathcal{E}[e_2]\rho'$$

$$\text{où } \rho' = \rho \oplus [f \mapsto \mathcal{E}[e_1]\rho']$$

$$\mathcal{E}[\text{fun } x \rightarrow e]\rho = f \text{ telle que } f(v) = \mathcal{E}[e](\rho \oplus [x \mapsto v])$$

Fonctions : construction de valeurs fonctionnelles

$\Rightarrow$ casse la récursion : retourne une fonction (dont le corps fera l'évaluation) mais n'évalue pas.

Récursion : construction récursive de la valeur sémantique

PCF avec affectations

Langage enrichi par

- des **références** (locations) correspondant aux variables
- une **mémoire** qui fait correspondre des valeurs à des locations

Syntaxe

$e \in \text{Exp}$, défini par $e ::= \dots \mid s; e$

$s \in \text{Stm}$, défini par $s ::= x := e \mid s_1; s_2$

Domaines sémantiques

$D = \text{INT} + \text{BOOL} + \text{FUN}$

avec $\text{FUN} = (D \times \text{STORE}) \rightarrow (D \times \text{STORE})$

et $\text{STORE} = \text{LOC} \rightarrow D$

$\text{ENV} = \text{ID} \rightarrow \text{LOC}$

LOC

- $\Rightarrow$ Modification sémantique du `let` : variables liées **mutables**

PCF avec affectations

Fonctions sémantiques

$$\mathcal{E} : \text{Exp} \rightarrow \text{ENV} \rightarrow \text{STORE} \rightarrow (D \times \text{STORE})$$

$$\mathcal{S} : \text{Stm} \rightarrow \text{ENV} \rightarrow \text{STORE} \rightarrow \text{STORE}$$

Définitions

$$\mathcal{E}[\![c]\!] \rho \sigma = (d_c, \sigma) \text{ pour chaque } c \in \text{Const}$$

$$\mathcal{E}[\![x]\!] \rho \sigma = (\sigma(\rho[\![x]\!])), \sigma), \text{ pour } x \in \text{Id}$$

$$\begin{aligned} \mathcal{E}[\![e_1 + e_2]\!] \rho \sigma = \\ \text{soit } (n_1, \sigma_1) &= \mathcal{E}[\![e_1]\!] \rho \sigma \\ \text{soit } (n_2, \sigma_2) &= \mathcal{E}[\![e_2]\!] \rho \sigma_1 \\ (n_1 + n_2, \sigma_2) \end{aligned}$$

PCF avec affectations

Définitions (suite)

$$\begin{aligned}\mathcal{E}[\![e_1 \ e_2]\!] \rho \sigma &= \\ \text{soit } (f, \sigma_1) &= \mathcal{E}[\![e_1]\!] \rho \sigma \\ \text{soit } (v, \sigma_2) &= \mathcal{E}[\![e_2]\!] \rho \sigma_1 \\ (f(v), \sigma_2) &\end{aligned}$$

$$\begin{aligned}\mathcal{E}[\![e_1 = e_2]\!] \rho \sigma &= \\ \text{soit } (v_1, \sigma_1) &= \mathcal{E}[\![e_1]\!] \rho \sigma \\ \text{soit } (v_2, \sigma_2) &= \mathcal{E}[\![e_2]\!] \rho \sigma_1 \\ \text{si } v_1 = v_2 \text{ alors } (true, \sigma_2) &\text{ sinon } (false, \sigma_2)\end{aligned}$$

PCF avec affectations

Définitions (suite)

$$\begin{aligned}\mathcal{E}[\text{if } e_1 \text{ then } e_2 \text{ else } e_3]\rho\sigma = \\ \text{soit } (v_1, \sigma_1) = \mathcal{E}[e_1]\rho\sigma \\ \text{if } v_1 = \text{true} \text{ then } \mathcal{E}[e_2]\rho\sigma_1 \text{ else } \mathcal{E}[e_3]\rho\sigma_1\end{aligned}$$

$$\begin{aligned}\mathcal{E}[\text{let } x = e_1 \text{ in } e_2]\rho\sigma = \\ \text{soit } (v_1, \sigma_1) = \mathcal{E}[e_1]\rho\sigma \\ \text{soit } h_1 \text{ une nouvelle référence} \\ \mathcal{E}[e_2](\rho \oplus [x \mapsto h_1])(\sigma_1 \oplus [h_1 \mapsto v_1])\end{aligned}$$

PCF avec affectations

Définitions (fin)

$$\mathcal{E}[\text{let rec } x = e_1 \text{ in } e_2]\rho\sigma =$$

soit l_1 une nouvelle référence

$$\text{soit } (v_1, \sigma_1) = \mathcal{E}[e_1](\rho \oplus [x \mapsto l_1])(\sigma)$$

$$\mathcal{E}[e_2](\rho \oplus [x \mapsto l_1])(\sigma_1 \oplus [l_1 \mapsto v_1])$$

$\sigma(l_1)$ est indéfinie : le calcul de e_1 ne doit pas utiliser x

$$\mathcal{E}[\text{fun } x \rightarrow e]\rho\sigma = f$$

telle que $f(v, \sigma') =$

soit l_x une nouvelle référence

$$\mathcal{E}[e](\rho \oplus [x \mapsto l_x])(\sigma' \oplus [l_x \mapsto v])$$

$\Rightarrow$ casse la récursion : retourne une fonction (dont le corps fera l'évaluation) mais n'évalue pas.

PCF avec affectations

Définitions (fin)

$$\begin{aligned}\mathcal{E}[[s; e]]\rho\sigma &= \\ \text{soit } \sigma' &= \mathcal{S}[[s]]\rho\sigma \\ \mathcal{E}[[e]]\rho\sigma' &\end{aligned}$$

$$\begin{aligned}\mathcal{S}[[x := e]]\rho\sigma &= \\ \text{soit } l_x &= \rho[[x]] \\ \text{soit } (v, \sigma') &= \mathcal{E}[[e]]\rho\sigma \\ \sigma' \oplus [l_x \mapsto v] &\end{aligned}$$

$$\begin{aligned}\mathcal{S}[[s_1; s_2]]\rho\sigma &= \\ \text{soit } \sigma_1 &= \mathcal{S}[[s_1]]\rho\sigma \\ \mathcal{S}[[s_2]]\rho\sigma_1 &\end{aligned}$$

PCF avec affectations et goto

Modéliser le calcul en identifiant le reste du calcul : la **continuation**

- construction goto : **changer le reste du calcul**
- déclaration d'étiquette : identifier ce point de programme comme continuation potentielle

C'est quoi, le calcul ?

- c'est transformer le « résultat » **courant** en résultat **final**
 - dépend de l'**état** courant de la mémoire
 - dépend de la **valeur** courante

⇒ chaque calcul doit être paramétré par « **ce qu'il reste à faire** »

Continuation (« ce qu'il reste à faire »)

- continuation d'instructions :

$$Cc = STORE \rightarrow STORE$$

- continuation d'expression :

$$Ec = (D \times STORE) \rightarrow (D \times STORE)$$

PCF avec affectations et goto

Domaines et fonctions sémantiques

$$\mathcal{E} : \text{Exp} \rightarrow \text{ENV} \rightarrow \text{STORE} \rightarrow \text{Ec} \rightarrow (\text{D} \times \text{STORE})$$

$$\mathcal{S} : \text{Stm} \rightarrow \text{ENV} \rightarrow \text{STORE} \rightarrow \text{Cc} \rightarrow \text{STORE}$$

Valeurs fonctionnelles reçoivent une **continuation**

$$\text{FUN} = (\text{D} \times \text{STORE}) \rightarrow \text{Ec} \rightarrow (\text{D} \times \text{STORE})$$

Les labels sont de simples identificateurs dont les valeurs (dans la **mémoire**) sont des continuations **d'instructions**. On aura donc :

$$\text{D} = \text{INT} + \text{BOOL} + \text{FUN} + \text{Cc}$$

PCF avec affectations et goto

Syntaxe

$s \in \text{Stm}$ et $s ::= x := e \mid s_1; s_2 \mid \textit{lab} : s \mid \mathbf{goto} \textit{lab}$

où

- $\textit{lab} \in \text{Id}$
- dans “ $\textit{lab} : s$ ”, s est la portée de $\textit{lab}$, et peut donc contenir “ $\mathbf{goto} \textit{lab}$ ”.

Définition

$\mathcal{E}[\![c]\!]\rho\sigma k = k(d_c, \sigma)$ pour $c \in \text{Const}$

$\mathcal{E}[\![x]\!]\rho\sigma k = k(\sigma(\rho[\![x]\!])), \sigma)$, pour $x \in \text{Id}$

PCF avec affectations et goto

Définition (suite)

$$\mathcal{E}[\![e_1 + e_2]\!] \rho \sigma k =$$

$$\mathcal{E}[\![e_1]\!] \rho \sigma k_1$$

$$\text{où } k_1 \text{ est définie par } k_1(n_1, \sigma_1) = \mathcal{E}[\![e_2]\!] \rho \sigma_1(k_2 \ n_1)$$

$$\text{où } k_2 \ n_1 \ (n_2, \sigma_2) = k(n_1 + n_2, \sigma_2)$$

$$\mathcal{E}[\![e_1 \ e_2]\!] \rho \sigma k =$$

$$\mathcal{E}[\![e_1]\!] \rho \sigma k_1$$

$$\text{où } k_1(v_1, \sigma_1) = \mathcal{E}[\![e_2]\!] \rho \sigma_1(k_2 \ v_1)$$

$$\text{où } k_2 \ v_1 \ (v_2, \sigma_2) = v_1 \ (v_2, \sigma_2) \ k$$

PCF avec affectations et goto

Définition (suite)

$\mathcal{E}[\![e_1 = e_2]\!] \rho \sigma k = \dots$ laissé en exercice

$\mathcal{E}[\![\text{if } e_1 \text{ then } e_2 \text{ else } e_3]\!] \rho \sigma k = \dots$ laissé en exercice

$\mathcal{E}[\![\text{let } x = e_1 \text{ in } e_2]\!] \rho \sigma k = \dots$ laissé en exercice

$\mathcal{E}[\![\text{let rec } x = e_1 \text{ in } e_2]\!] \rho \sigma k = \dots$ laissé en exercice

$\mathcal{E}[\![\text{fun } x \rightarrow e]\!] \rho \sigma k =$

$k(f, \sigma)$

où f est définie par

$$f(v, \sigma') \text{ } k' = \mathcal{E}[\![e]\!] (\rho \oplus [x \mapsto l_x]) (\sigma' \oplus [l_x \mapsto v]) k'$$

où l_x est une nouvelle référence

PCF avec affectations et goto

Définition (suite)

$$\begin{aligned}\mathcal{E}[\![s; e]\!] \rho \sigma k &= \\ \text{soit } k' \text{ définie par } k'(\sigma') &= \mathcal{E}[\![e]\!] \rho \sigma' k \\ \mathcal{S}[\![s]\!] \rho \sigma k' &\end{aligned}$$

$$\begin{aligned}\mathcal{S}[\![lab : s]\!] \rho \sigma k &= \\ \text{soit } l \text{ une nouvelle référence} & \\ \text{soit } k' \text{ définie récursivement par} & \\ k'(\sigma') &= \mathcal{S}[\![s]\!] (\rho \oplus [lab \mapsto l]) (\sigma' \oplus [l \mapsto k']) k \\ k' \sigma &\end{aligned}$$

k' : « ce qu'il reste à faire », càd évaluer s
 $\Rightarrow$ évaluation de s peut faire un goto vers s

PCF avec affectations et goto

Définition (fin)

$$\begin{aligned} \mathcal{S}[\![x := e]\!] \rho \sigma k &= \\ \text{soit } k' \text{ définie par } k'(v', \sigma') &= k(\sigma' \oplus [\rho[\![x]\!] \mapsto v']) \\ \mathcal{E}[\![e]\!] \rho \sigma k' \end{aligned}$$

$$\begin{aligned} \mathcal{S}[\![s_1; s_2]\!] \rho \sigma k &= \\ \mathcal{S}[\![s_1]\!] \rho \sigma k' \\ \text{où } k' \text{ est définie par } k'(\sigma') &= \mathcal{S}[\![s_2]\!] \rho \sigma' k \end{aligned}$$

$$\begin{aligned} \mathcal{S}[\![\text{goto } lab]\!] \rho \sigma k &= \\ k' \sigma \\ \text{où } k' &= \sigma(\rho[\![lab]\!]) \end{aligned}$$

Conclusion

Vision mathématique

Ignore le calcul

Styles fonctionnels

- utilisation de **fonctions sémantiques** : $\mathcal{S}$ et $\mathcal{E}$
- continuations sont des fonctions

À mettre en œuvre dans un langage fonctionnel

- nécessité de créer des fonctions comme valeurs (fermetures)

Principes des lang. de progr. INE 213

François Pessaux
(Sur la base d'un cours de Michel Mauny)

ENSTA, édition 2025-2026

`prénom.nom@ensta.fr`



Sémantique opérationnelle

- 1 Sémantique opérationnelle
 - Évaluation stricte du noyau fonctionnel d'un petit langage
 - Ajout de données structurées
- 2 Interprétation
- 3 Compilation

Sémantique dénotationnelle vs. opérationnelle

Sémantique dénotationnelle

- valeurs **abstraites**
- le sens, **pas le calcul**
- quid des programmes qui ne terminent pas ?
- quid des programmes qui produisent une erreur d'exécution ?

Sémantique opérationnelle

- utilise des valeurs **concrètes** (syntaxiques)
- explicite les **étapes du calcul**
- procède par **réduction** (transformation syntaxique) du programme initial
- décrite par des **catégories syntaxiques** + des **règles d'inférence**
- G. Plotkin, 1981 ; G. Kahn 1985 (sémantique « naturelle »)

Noyau fonctionnel : évaluation stricte

Le langage

$c \in \text{Const}$ *Constantes, incluant les fonctions primitives*
 $x \in \text{Id}$ *Identificateurs*
 $e \in \text{Exp}$ *Expressions*

Syntaxe

$e ::= c \mid x \mid e_1 + e_2 \mid e_1 = e_2$
 $\mid$ **if** e_1 **then** e_2 **else** e_3
 $\mid$ $e_1 \ e_2$
 $\mid$ **fun** $x \rightarrow e$
 $\mid$ **let** $x = e_1$ **in** e_2
 $\mid$ **let rec** $f(x) = e_1$ **in** e_2

Noyau fonctionnel, évaluation stricte

Valeurs

$$v ::= c \mid \text{Valf}(x, e, \rho) \mid \text{Valfr}(f, x, e, \rho)$$

- ρ environnement : $x \in Id \rightarrow v$
- $\rho \oplus [x \mapsto v]$: extension de ρ
 - $(\rho \oplus [x \mapsto v])(x) = v$
 - $(\rho \oplus [x \mapsto v])(y) = \rho(y)$ si $y \neq x$
- Valf et Valfr : valeurs fonctionnelles (**fermetures** ou *closures*)

Réponses

$$r ::= v \mid \text{Erreur}$$

Règles d'évaluation

définissent des « jugements » de la forme

$$\rho \vdash e \Rightarrow r$$

« Dans l'env. ρ , l'évaluation de l'expression e produit la réponse r . »

Langage fonctionnel, strict : règles d'inférence

1/4

Constantes, variables, fonctions

$$\rho \vdash c \Rightarrow c \text{ (Const)} \qquad \frac{x \in \text{dom}(\rho)}{\rho \vdash x \Rightarrow \rho(x)} \text{ (Ident)}$$

$$\rho \vdash (\text{fun } x \rightarrow e) \Rightarrow \text{Valf}(x, e, \rho) \text{ (Fun)}$$

Conditionnelle

$$\frac{\rho \vdash e_1 \Rightarrow T \quad \rho \vdash e_2 \Rightarrow r}{\rho \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow r} \text{ (IfTrue)}$$

$$\frac{\rho \vdash e_1 \Rightarrow F \quad \rho \vdash e_3 \Rightarrow r}{\rho \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow r} \text{ (IfFalse)}$$

Langage fonctionnel, strict : règles

2/4

Primitives

$$\frac{\rho \vdash e_1 \Rightarrow n_1 \quad \rho \vdash e_2 \Rightarrow n_2 \quad n = n_1 + n_2}{\rho \vdash (e_1 + e_2) \Rightarrow n} \text{ (Plus)}$$

$$\frac{\rho \vdash e_1 \Rightarrow v_1 \quad \rho \vdash e_2 \Rightarrow v_2 \quad v_1 = v_2}{\rho \vdash (e_1 = e_2) \Rightarrow T} \text{ (EgTrue)}$$

$$\frac{\rho \vdash e_1 \Rightarrow v_1 \quad \rho \vdash e_2 \Rightarrow v_2 \quad v_1 \neq v_2}{\rho \vdash (e_1 = e_2) \Rightarrow F} \text{ (EgFalse)}$$

Langage fonctionnel, strict : règles

3/4

Déclarations

$$\frac{\rho \vdash e_1 \Rightarrow v_1 \quad \rho \oplus [x \mapsto v_1] \vdash e_2 \Rightarrow r}{\rho \vdash (\text{let } x = e_1 \text{ in } e_2) \Rightarrow r} \text{ (Let)}$$

$$\frac{\rho \oplus [f \mapsto \text{Valfr}(f, x, e_1, \rho)] \vdash e_2 \Rightarrow r}{\rho \vdash (\text{let rec } f(x) = e_1 \text{ in } e_2) \Rightarrow r} \text{ (Letrec)}$$

Langage fonctionnel, strict : règles

4/4

Applications

$$\frac{\rho \vdash e_1 \Rightarrow \text{Valf}(x, e_0, \rho_0) \quad \rho \vdash e_2 \Rightarrow v_2 \quad \rho_0 \oplus [x \mapsto v_2] \vdash e_0 \Rightarrow r}{\rho \vdash (e_1 e_2) \Rightarrow r} \text{ (AppF)}$$

$$\frac{\rho \vdash e_1 \Rightarrow \text{Valfr}(f, x, e_0, \rho_0) \quad \rho \vdash e_2 \Rightarrow v_2 \quad \rho_0 \oplus [f \mapsto \text{Valfr}(f, x, e_0, \rho_0)] \oplus [x \mapsto v_2] \vdash e_0 \Rightarrow r}{\rho \vdash (e_1 e_2) \Rightarrow r} \text{ (AppFR)}$$

Les règles auxquelles vous avez échappé

Production d'erreurs et propagation

$$\frac{x \notin \text{dom}(\rho)}{\rho \vdash x \Rightarrow \text{Erreur}} \quad (\text{IdentErr})$$

$$\frac{\rho \vdash e_1 \Rightarrow r \notin \{T, F\}}{\rho \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow \text{Erreur}} \quad (\text{IfErr})$$

$$\frac{\rho \vdash e_1 \Rightarrow r \notin \mathbb{N}}{\rho \vdash (e_1 + e_2) \Rightarrow \text{Erreur}} \quad (\text{PlusErrL})$$

$$\frac{\rho \vdash e_1 \Rightarrow n_1 \quad \rho \vdash e_2 \Rightarrow r \notin \mathbb{N}}{\rho \vdash (e_1 + e_2) \Rightarrow \text{Erreur}} \quad (\text{PlusErrR})$$

$$\frac{\rho \vdash e_1 \Rightarrow \text{Erreur}}{\rho \vdash (e_1 = e_2) \Rightarrow \text{Erreur}} \quad (\text{EqErrL})$$

$$\frac{\rho \vdash e_1 \Rightarrow v_1 \quad \rho \vdash e_2 \Rightarrow \text{Erreur}}{\rho \vdash (e_1 = e_2) \Rightarrow F} \quad (\text{EqErrR})$$

$$\frac{\rho \vdash e_1 \Rightarrow \text{Erreur}}{\rho \vdash (\text{let } x = e_1 \text{ in } e_2) \Rightarrow \text{Erreur}} \quad (\text{LetErr})$$

$$\frac{\rho \vdash e_1 \Rightarrow r \neq \text{Valf}(_, _, _) \wedge r \neq \text{Valfr}(_, _, _, _)}{\rho \vdash (e_1 e_2) \Rightarrow \text{Erreur}} \quad (\text{AppErrL})$$

$$\frac{\rho \vdash e_1 \Rightarrow \text{Valf}(x, e_0, \rho_0) \quad \rho \vdash e_2 \Rightarrow \text{Erreur}}{\rho \vdash (e_1 e_2) \Rightarrow \text{Erreur}} \quad (\text{AppFErrR})$$

$$\frac{\rho \vdash e_1 \Rightarrow \text{Valfr}(f, x, e_0, \rho_0) \quad \rho \vdash e_2 \Rightarrow \text{Erreur}}{\rho \vdash (e_1 e_2) \Rightarrow \text{Erreur}} \quad (\text{AppFErrR})$$

Arbres d'évaluation – dérivations

Arbres : **axiomes** = feuilles, **règles d'inférence** = nœuds

Posons $\bar{f} = \text{Valf}(x, x, \emptyset)$ et $\rho_f = \emptyset \oplus [f \mapsto \bar{f}] = [f \mapsto \bar{f}]$.

$$\frac{\frac{f \in \text{dom}(\rho_f)}{\rho_f \vdash f \Rightarrow \bar{f}} \quad \rho_f \vdash 1 \Rightarrow 1 \quad \frac{x \in \text{dom}(\emptyset \oplus [x \mapsto 1])}{\emptyset \oplus [x \mapsto 1] \vdash x \Rightarrow 1}}{\rho_f \vdash f(1) \Rightarrow 1} \quad (\text{App})$$

$$\frac{(\text{Fun}) \quad \overline{\emptyset \vdash (\text{fun } x \rightarrow x) \Rightarrow \bar{f}}}{\emptyset \vdash (\text{let } f = \text{fun } x \rightarrow x \text{ in } f(1)) \Rightarrow 1} \quad (\text{Let})$$

Données structurées

Couples et projections

$e ::= \dots \mid (e_1, e_2) \mid \text{fst } e \mid \text{snd } e$

Valeurs

$v ::= \dots \mid (v_1, v_2)$

Règles

$$\frac{\rho \vdash e_1 \Rightarrow v_1 \quad \rho \vdash e_2 \Rightarrow v_2}{\rho \vdash (e_1, e_2) \Rightarrow (v_1, v_2)}$$

$$\frac{\rho \vdash e \Rightarrow (v_1, v_2)}{\rho \vdash \text{fst } e \Rightarrow v_1}$$

$$\frac{\rho \vdash e \Rightarrow (v_1, v_2)}{\rho \vdash \text{snd } e \Rightarrow v_2}$$

De l'évaluation (formelle) à l'interprétation (mécanique)

Implémenter les valeurs de la sémantique opérationnelle :

$$v ::= c \mid \text{Valf}(x, e, \rho) \mid \text{Valfr}(f, x, e, \rho)$$

par :

```
type semopval =  
  | Intval of int  
  | Boolval of bool  
  | Stringval of string  
  | ...  
  | Funval of { param: string; body: expr; env: environment }  
  | Funrecval of  
    { fname: string; param: string; body: expr; env: environment }  
  
and environment = (string * semopval) list ;;
```

On considère qu'une *réponse* sera ou bien une valeur, ou alors la levée d'une **exception OCaml**.

De l'évaluation à l'interprétation

Puis « lire » les axiomes et règles d'inférence comme des cas d'une fonction d'évaluation :

```
val eval : Pcfast.expr → environment → semopval
```

```
let rec eval t rho = match t with ...
```

```
| Pcfast.EInt n → Intval n                                (*  $\rho \vdash c \Rightarrow c$  *)  
| Pcfast.EString s → Stringval s                          (*  $\rho \vdash c \Rightarrow c$  *)  
  
| Pcfast.EIdent x → begin  
    try List.assoc x rho with                               (*  $\frac{x \in \text{dom}(\rho)}{\rho \vdash x \Rightarrow \rho(x)}$  *)  
    Not_found →  
        error (Printf.sprintf "Unbound variable %s." x)  
    end  
| ...
```

De l'évaluation à l'interprétation

Les axiomes et règles d'inférence comme des cas d'une fonction d'évaluation :

let rec eval t rho = match t with ...

```

| Pcfast.EFun (x, e) →  $(\ast \rho \vdash (\text{fun } x \rightarrow e) \Rightarrow \text{Valf}(x, e, \rho) \ast)$ 
    Funval { param = x; body = e; env = rho }
| Pcfast.EIf (e1, e2, e3) → begin
    match eval e1 rho with
    | Boolval true → eval e2 rho
    | Boolval false → eval e3 rho  $(\ast \frac{\rho \vdash e_1 \Rightarrow T \quad \rho \vdash e_2 \Rightarrow r}{\rho \vdash \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \Rightarrow r} \ast)$ 
    | _ → error "Bool value expected"
end
| ...

```

De l'évaluation à l'interprétation

Les axiomes et règles d'inférence comme des cas d'une fonction d'évaluation :

```
let rec eval t rho = match t with ...
```

```
| Pcfast.EBinop ("+", e1, e2) → begin (*  $\frac{\rho \vdash e_1 \Rightarrow n_1 \quad \rho \vdash e_2 \Rightarrow n_2}{\rho \vdash (e_1 + e_2) \Rightarrow n_1 + n_2}$  *)  
  match (eval e1 rho, eval e2 rho) with  
  | (Intval n1, Intval n2) → Intval (n1 + n2)  
  | _ → error "Integer values expected"  
end  
  
| ...
```


De l'évaluation à l'interprétation

Les axiomes et règles d'inférence comme des cas d'une fonction d'évaluation :

let rec eval t rho = match t with ...

Pcfast.ELet (x, e ₁ , e ₂) →	$\left(* \frac{\rho \vdash e_1 \Rightarrow v_1 \quad \rho \oplus [x \mapsto v_1] \vdash e_2 \Rightarrow r}{\rho \vdash (\text{let } x = e_1 \text{ in } e_2) \Rightarrow r} * \right)$
let v ₁ = eval e ₁ rho in eval e ₂ ((x,v ₁)::rho)	
Pcfast.ELetrec (f, x, e ₁ , e ₂) →	$\left(* \frac{\rho \oplus [f \mapsto \text{Valfr}(f, x, e_1, \rho)] \vdash e_2 \Rightarrow r}{\rho \vdash (\text{let rec } f(x) = e_1 \text{ in } e_2) \Rightarrow r} * \right)$
let v _f = Funrecval { fname = f; param = x; body = e ₁ ; env = rho } in eval e ₂ ((f, v _f)::rho)	
...	

De l'évaluation à l'interprétation

Des axiomes et règles d'inférence aux cas d'une fonction d'évaluation

```
let rec eval t rho = match t with
```

```
...
```

```
| Pcfast.EApp (e1, e2) →
```

```
  let (vf, v2) = (eval e1 rho, eval e2 rho) in begin
```

```
  match vf with
```

$$\left(* \frac{\rho \vdash e_1 \Rightarrow \text{Valf}(x, e_0, \rho_0) \quad \rho \vdash e_2 \Rightarrow v_2 \quad \rho_0 \oplus [x \mapsto v_2] \vdash e_0 \Rightarrow r}{\rho \vdash (e_1 e_2) \Rightarrow r} * \right)$$

```
| Funval { param = x; body = e0; env = rho0 } →  
  eval e0 ((x, v2) :: rho0)
```

$$\left(* \frac{\rho \vdash e_1 \Rightarrow \text{Valfr}(f, x, e_0, \rho_0) \quad \rho \vdash e_2 \Rightarrow v_2 \quad \rho_0 \oplus [f \mapsto \text{Valfr}(f, x, e_0, \rho_0)] \oplus [x \mapsto v_2] \vdash e_0 \Rightarrow r}{\rho \vdash (e_1 e_2) \Rightarrow r} * \right)$$

```
| Funrecval { fname = f; param = x; body = e0; env=rho0 } →  
  eval e0 ((x, v2) :: (f, vf) :: rho0)
```

```
| _ → error "Expecting a functional value"
```

```
end ;;
```

De l'interprétation à la compilation

Interprète

- + environnements, valeurs
- gestion du contrôle par le langage hôte
 - $e_1 + e_2 \rightsquigarrow (\text{eval } e_1) + (\text{eval } e_2)$: ordre eval's non spécifié

(Un modèle de) Compilateur

- + environnements, valeurs
- + gestion explicite du contrôle (pile)
 - $e_1 + e_2 \rightsquigarrow r_1 \leftarrow (\text{eval } e_2); r_2 \leftarrow (\text{eval } e_1); r_3 \leftarrow \text{add } r_1, r_2$

Machine abstraite

- code, pile, registre, (mémoire, etc.)
- instruction : état $\rightarrow$ état

Compilation

Le langage

$$e ::= c \mid x \mid e_1 e_2 \mid \text{fun } x \rightarrow e \mid \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \\ \mid e_1 + e_2 \mid e_1 = e_2 \mid \text{let } x = e_1 \text{ in } e_2$$

Codage de *de Bruijn* des occurrences de variables

- occurrence de x dans un contexte où n lieux («**fun** $y \rightarrow$ », «**let** $y =$ » ou «**let** $f(y) =$ ») séparent cette occurrence de son lieu
- correspond à un accès à une profondeur n dans un environnement : $\text{Access}(n)$

Exemples

fun $x \rightarrow x$

$\text{Access}(0)$

fun $x \rightarrow \text{let } y = \dots \text{ in let } z = \dots \text{ in fun } t \rightarrow \dots x \dots$

$\text{Access}(3)$

Compilation

État de la machine (registre, code, pile)

- **registre** : contient une **valeur** ou un **environnement**
- **code** : instructions
- **pile** : chaque élément est **valeur**, ou **env.**, ou **adresse de retour**

État initial / début de fonction : registre contient l'environnement.

Les instructions

$i ::=$	Loadi (n) Loadb (b)	chargement de cst dans le registre
	Plus Equal	opérations binaires (arith, tests, ...)
	Access (n)	accès dans env.
	Branch (c_1, c_2)	branchement
	Push Swap	empile, échange
	Mkclos (c)	constr. de valeur fonctionnelle
	Apply	application

La machine dans tous ses états

État	État suivant
$(r, \text{Loadi } (n) :: c, p)$	$\Rightarrow (n, c, p)$
$(r, \text{Loadb } (b) :: c, p)$	$\Rightarrow (b, c, p)$
$(n, \text{Plus } :: c, m :: p)$	$\Rightarrow (\overline{n + m}, c, p)$
$(n, \text{Equal } :: c, m :: p)$	$\Rightarrow (\overline{n = m}, c, p)$
$(\text{true}, \text{Branch } (c_1, c_2) :: c, r :: p)$	$\Rightarrow (r, c_1, c :: p)$
$(\text{false}, \text{Branch } (c_1, c_2) :: c, r :: p)$	$\Rightarrow (r, c_2, c :: p)$
$(r, \text{Push } :: c, p)$	$\Rightarrow (r, c, r :: p)$
$(r_1, \text{Swap } :: c, r_2 :: p)$	$\Rightarrow (r_2, c, r_1 :: p)$
$(r, \text{Mkclos } (c_1) :: c, p)$	$\Rightarrow (\langle c_1, r \rangle, c, p)$
$(v, \text{Apply } :: c, \langle c_0, r_0 \rangle :: p)$	$\Rightarrow ((v \oplus r_0), c_0, c :: p)$
$(\rho, \text{Access } (n) :: c, p)$	$\Rightarrow (\overline{\rho(n)}, c, p)$
$(r, [], c :: p)$	$\Rightarrow (r, c, p)$

La machine dans tous ses états (en OCaml)

```
let next state = match state with
```

```
| (r,      Loadi n::c,      p)      → (I(n), c, p)
| (r,      Loadb b::c,      p)      → (B(b), c, p)
| (I(n),    Plus::c,        I(m)::p) → (I(n+m), c, p)
| (I(n),    Equal::c,       I(m)::p) → (B(n=m), c, p)
| (B(true), Branch(c1,c2)::c, r::p) → (r, c1, A(c)::p)
| (B(false), Branch(c1,c2)::c, r::p) → (r, c2, A(c)::p)
| (r,      Push::c,         p)      → (r, c, r::p)
| (r1,     Swap::c,        r2::p) → (r2, c, r1::p)
| (r,      Mkclos(c1)::c,   p)      → (C(c1, r), c, p)
| (v,      Apply::c,        C(c0,r0)::p) → (E(v,r0), c0, A(c)::p)
| (E(v::_), Access(0)::c,    p)      → (v, c, p)
| (E(v::q), Access(n)::c,    p)      → (q, Access(n-1)::c, p)
| (r,      [ (* ret *) ],    A(c)::p) → (r, c, p)
| (r,      [ ],              [ ])    → raise (Success r)
| _ → error "Error: invalid state (machine stopped)"
```

La compilation

Compilation du noyau

$\llbracket n \rrbracket_\rho = \text{Loadi } (n)$

$\llbracket b \rrbracket_\rho = \text{Loadb } (b)$

$\llbracket x \rrbracket_\rho = \text{Access } (n)$

où n est la profondeur de x dans ρ

$\llbracket e_1 e_2 \rrbracket_\rho = \text{Push}; \llbracket e_1 \rrbracket_\rho; \text{Swap}; \llbracket e_2 \rrbracket_\rho; \text{Apply}$

$\llbracket \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \rrbracket_\rho$
 $= \text{Push}; \llbracket e_1 \rrbracket_\rho; \text{Branch } (\llbracket e_2 \rrbracket_\rho, \llbracket e_3 \rrbracket_\rho)$

$\llbracket \text{fun } x \rightarrow e \rrbracket_\rho = \text{Mkclos } (\llbracket e \rrbracket_{x, \rho})$

$\llbracket \text{let } x = e_1 \text{ in } e_2 \rrbracket_\rho = \llbracket (\text{fun } x \rightarrow e_2) e_1 \rrbracket_\rho$

Compilation des définitions récursives ?

- Laissez en exercice (pas si simple)

Conclusion

La sémantique opérationnelle

- précise **comment** s'effectuent les calculs
- se rapproche des **interprètes**

La compilation

- consiste en premier lieu à **explicitement le contrôle**
- ⇒ manipulation de pile
- Nombreux autres traitements non abordés :
 - allocation de registres,
 - optimisations,
 - sélection d'instructions (production de code assembleur),
 - ...

Principes des lang. de progr. INE 213

François Pessaux
(Sur la base d'un cours de Michel Mauny)

ENSTA, édition 2025-2026

`prénom.nom@ensta.fr`



Termes

- 1 Termes du premier ordre
 - Termes sans variables
 - Définitions inductives
 - Termes avec variables
- 2 Filtrage
- 3 Unification

Motivations

Termes

- représentation d'**expressions** quelconques
 - $1 + 2$
 - **if** $p(x)$ **then** $g(f(x), h(x, y, z))$ **else** $\text{succ}(1+t)$
 - $P(x) \vee \neg Q(x)$
- utilité : logique, types, données structurées, morceaux de programme
- ajout de **variables** pour représenter une **infinité** de termes

Du 1^{er} ordre : sans liaisons de variables

- **oui** : $P(x) \vee \neg Q(x)$
- **non** : $\forall x, P(x) \vee \neg Q(x)$

Termes

Symboles : on considère Σ , un ensemble **symboles** munis d'une **arité**

- $\Sigma_1 = \{(\text{Zero}, 0), (\text{Succ}, 1), (\text{Plus}, 2)\}$
- $\Sigma_2 = \{(\text{A}, 0), (\text{F}, 1), (\text{G}, 3)\}$

Définition

Termes

L'ensemble $\mathcal{T}_\Sigma$ des **termes sur Σ** (appelés Σ -termes) est défini par :

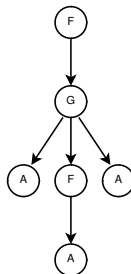
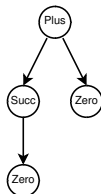
- tout symbole de Σ d'arité 0 est un Σ -terme
- si $t_1, \dots, t_n$ sont des Σ -termes et si $F \in \Sigma$ est un symbole d'arité n , alors $F(t_1, \dots, t_n)$ est un Σ -terme.

Exemples

Exemples

- $\text{Plus} (\text{Succ} (\text{Zero}), \text{Zero}) \in \mathcal{T}_{\Sigma_1}$
- $F (G (A, F (A), A)) \in \mathcal{T}_{\Sigma_2}$
- $\text{Succ} (F (A)) \in \mathcal{T}_{\Sigma_1 \cup \Sigma_2}$
- **Zero (Succ)** n'est pas un terme valide

Graphiquement



Définitions inductives

Ensembles de termes quelquefois définis **inductivement**

Définition inductive de $\mathcal{I}$, sous-ensemble de $\mathcal{E}$ donnée par

- sous-ensemble fini $\mathcal{B}$ (« base ») de $\mathcal{I}$
- sous-ensemble fini $\mathcal{C}$ de constructeurs $C_i : \mathcal{I}^{n_i} \rightarrow \mathcal{I}$

Plus petit ensemble contenant une relation

- $\mathcal{B} \subseteq \mathcal{I}$
- $\forall C \in \mathcal{C}, \forall x_1 \dots x_n \in \mathcal{I}, C(x_1, \dots, x_n) \in \mathcal{I}$

Exemple

- $\text{nat} = \text{Zero} \mid \text{Succ of nat}$
- que l'on peut lire comme
nat est *le plus petit ensemble tel que :*
 - $\text{Zero} \in \text{nat}$
 - si $n \in \text{nat}$ alors $\text{Succ}(n) \in \text{nat}$

Définitions inductives

On retrouve de telles définitions dans :

- définitions de type OCaml (types dits *algébriques*)
- définitions de grammaires *algébriques*
- ...

Entiers naturels

Pairs

- 0 est pair
- n pair $\Rightarrow n + 2$ pair

Impairs

- 1 est impair
- n impair $\Rightarrow n + 2$ impair

Preuves par induction

Soit E défini par induction

- avec cas de base $b_1, \dots, b_n$
- cas d'induction $p_i \Rightarrow q_i$

Pour montrer P sur E

- montrer $P(b_i)$
- montrer $P(p_i) \Rightarrow P(q_i)$

généralisation de la récurrence (qui porte sur les entiers)

Exemple

- $\text{arbre} = F \mid N \text{ of } (\text{arbre} * \text{arbre})$
- montrer que le nombre de feuilles d'un arbre est au plus égal à 1 + le nombre de nœuds ($f_a \leq n_a + 1$)
- par **induction** sur la structure d'arbre

Preuves par induction

Montrer que le nombre de feuilles d'un arbre est au plus égal à
 $1 + \text{le nombre de nœuds } (f_a \leq n_a + 1)$

- par **induction** sur la structure d'arbre...
- cas $a = F$
 - (1) $f_a = 1$
 - (2) $n_a = 0$
 - Par (1) et (2), $1 \leq 0 + 1$ donc $f_a \leq n_a + 1$
- cas $a = N(t, u)$
 - (1) hypothèse d'induction : $f_t \leq n_t + 1$
 - (2) hypothèse d'induction : $f_u \leq n_u + 1$
 - (3) Par structure arbre : $n_a = n_t + n_u + 1$
 - (4) Par structure arbre : $f_a = f_t + f_u$
 - (5) Par (1), (2) et (4) : $f_a \leq n_t + n_u + 2$
 - (6) Par (5) et (3) : $f_a \leq n_a + 1$



Termes avec variables

$\mathcal{T}_\Sigma$ ne contient que des termes dits **clos**

Soit $\mathcal{V}$ ensemble infini dénombrable de noms de variables

Termes avec variables

- toute variable de $\mathcal{V}$ est un $\Sigma \cup \mathcal{V}$ -terme
- tout symbole de Σ d'arité 0 est un $\Sigma \cup \mathcal{V}$ -terme
- si $t_1, \dots, t_n$ sont des $\Sigma \cup \mathcal{V}$ -termes et si $F \in \Sigma$ est un symbole d'arité n , alors $F(t_1, \dots, t_n)$ est un $\Sigma \cup \mathcal{V}$ -terme

Dans la suite Σ et $\mathcal{V}$ sont implicites.

Substitution

Définition

Une **substitution** θ est une fonction totale de l'ensemble des **variables** $\mathcal{V}$ vers un ensemble de **termes** $\mathcal{T}$.

Domaine

- $\text{dom}(\theta)$ ensemble des x tels que $\theta(x) \neq x$
 - ▶ i.e « ensemble des variables modifiées par θ »
- on ne considérera que des substitutions à domaine **fini**

Notations, terminologie

- $[x_1 \mapsto t_1; \dots; x_n \mapsto t_n]$
- substitution **identité** : $\emptyset$ ou $[]$
- renommage : θ bijective dont le codomaine ne contient que des variables
 - ▶ i.e. « renomme des variables que **par des variables** »

Morphisme

De substitution à morphisme

On étend naturellement une substitution θ en un **morphisme** de termes $\bar{\theta}$:

- $\bar{\theta}(F(t_1, \dots, t_n)) = F(\bar{\theta}(t_1), \dots, \bar{\theta}(t_n))$
- $\bar{\theta}(x) = \theta(x)$

$\Rightarrow$ devient une fonction des **termes** vers les termes

Par la suite, $\bar{\theta}$ sera noté θ : on confondra substitution et morphisme associé.

Extension, composition

Composition

On définit la **composition** $\theta_1 \circ \theta_2$ de deux substitutions θ_1 et θ_2 comme la substitution qui associe à toute variable x le terme $\theta_1(\theta_2(x))$.

Extension de θ

L'**extension** de la substitution θ par $[x \mapsto t]$ est la substitution notée $\theta \oplus [x \mapsto t]$ définie par :

- $\text{dom}(\theta \oplus [x \mapsto t]) = \text{dom}(\theta) \cup \{x\}$
- $(\theta \oplus [x \mapsto t])(x) = t$
- $(\theta \oplus [x \mapsto t])(y) = \theta(y)$ pour $y \neq x$

Pré-ordre sur les termes

Les substitutions induisent un pré-ordre sur les termes

préordre = relation binaire réflexive et transitive

Préordre de généralité

On dira que t_1 est **moins général** que t_2 , et on notera $t_1 \leq t_2$ s'il existe une substitution θ telle que $t_1 = \theta(t_2)$.

On dira que t_1 est une **instance** de t_2 .

Filtrage

Le problème du filtrage : trouver une réponse à la question suivante :

Étant donnés deux termes t et m , a-t-on $t \leq m$?

que l'on peut aussi formuler comme :

Le terme t est-il une instance du terme m ?

Utilisation

- filtrage (*pattern matching*) des langages comme OCaml
- transposer un théorème général dans un contexte particulier

⇒ **Calcul de substitution** θ telle que $\theta(m) = t$

Exemples

terme	instance de ?	motif	substitution
$\text{Succ}(\text{Succ}(\text{Zero}))$	$\leq$	$\text{Succ}(x)$	$\theta = [x \mapsto \text{Succ}(\text{Zero})]$
$\text{Succ}(\text{Succ}(y))$	$\leq$	$\text{Succ}(x)$	$\theta = [x \mapsto \text{Succ}(y)]$
$\text{Succ}(\text{Succ}(x))$	$\leq$	$\text{Succ}(x)$	$\theta = [x \mapsto \text{Succ}(x)]$
$G(\text{F}(\text{A}), \text{F}(\text{A}), \text{F}(\text{A}))$	$\leq$	$G(x, x, x)$	$\theta = [x \mapsto \text{F}(\text{A})]$
$\text{Succ}(x)$	$\not\leq$	$\text{Succ}(\text{Succ}(y))$	—
Par contre, $\text{Succ}(\text{Succ}(y)) \leq \text{Succ}(x)$			
$G(\text{F}(\text{A}), \text{A}, \text{F}(\text{A}))$	$\not\leq$	$G(x, x, x)$	—
Et d'ailleurs, $G(x, x, x) \not\leq G(\text{F}(\text{A}), \text{A}, \text{F}(\text{A}))$ aussi			

Algorithme

filtre $[(m, t)] \emptyset$ est défini par

filtre : (terme $\times$ terme) list $\rightarrow$ subst. $\rightarrow$ subst. (ou échec)

filtre $[] \theta = \theta$

filtre $[(x, t_1); (m_2, t_2); \dots; (m_n, t_n)] \theta =$

si $x \notin \text{dom}(\theta)$ alors

filtre $[(m_2, t_2); \dots; (m_n, t_n)] (\theta \oplus [x \mapsto t_1])$

sinon si $\theta(x) = t_1$ alors filtre $[(m_2, t_2); \dots; (m_n, t_n)] \theta$

sinon échec

filtre $[(S(q_1, \dots, q_k), T(u_1, \dots, u_l)); (m_2, t_2); \dots; (m_n, t_n)] \theta =$

si $S = T$ alors filtre $[(q_1, u_1); \dots; (q_k, u_k); (m_2, t_2); \dots; (m_n, t_n)] \theta$

sinon échec

filtre n'importe quoi d'autre = échec

Unification

L'unification

- deux termes ont-ils des **instances communes** ?
- la réponse peut être :
 - « non, évidemment » : A et B , $F(\dots)$ et $G(\dots)$
 - « oui, évidemment » : $\text{Succ}(x)$ et $\text{Succ}(\text{Zero})$
 - moins évidente : $\text{Plus}(x, \text{Succ}(\text{Zero}))$ et $\text{Plus}(\text{Succ}(y), z)$
- résolution d'une équation entre termes $t_1 = t_2$, qui revient à calculer une substitution θ telle que $\theta(t_1) = \theta(t_2)$

Si deux termes admettent un **unificateur** θ , on dit qu'ils sont **unifiables**

Exemples

Termes à unifier	
$\text{Succ}(\text{Succ}(x))$ et $\text{Succ}(\text{Succ}(x))$	$\emptyset$
$\text{Plus}(\text{Zero}, \text{Zero})$ et $\text{Plus}(x, \text{Zero})$	$[x \mapsto \text{Zero}]$
$\text{Plus}(\text{Zero}, x)$ et $\text{Plus}(x, \text{Zero})$	$[x \mapsto \text{Zero}]$
$\text{Plus}(\text{Succ}(y), x)$ et $\text{Plus}(\text{Succ}(\text{Succ}(x)), \text{Succ}(\text{Zero}))$	$[y \mapsto \text{Succ}(\text{Succ}(\text{Zero})); x \mapsto \text{Succ}(\text{Zero})]$
$\text{Succ}(\text{Zero})$ et $\text{Succ}(\text{Succ}(\text{Zero}))$	—
$\text{Succ}(x)$ et $\text{Succ}(\text{Succ}(x))$	—

Dernier exemple : équation récursive

Principe de l'algorithme

Cas général

- soit à unifier $[(t_1, u_1), \dots, (t_n, u_n)]$: on cherche μ telle que $\forall i = 1..n, \mu(t_i) = \mu(u_i)$
- si t_1 est unifiable avec u_1 , produisant μ_1 , on continue l'unification de $[(\mu_1(t_2), \mu_1(u_2)), \dots, (\mu_1(t_n), \mu_1(u_n))]$
- si cela réussit et produit μ' , alors $\mu' \circ \mu_1$ unifie $[(t_1, u_1), \dots, (t_n, u_n)]$.

Algorithme

unifier $[(t, u)] \emptyset$ est défini par

unifier : (term $\times$ term) list $\rightarrow$ subst. $\rightarrow$ subst. (ou échec)

unifier $[] \mu = \mu$

(* Cas variable / terme. *)

unifier $[(x, u_1); (t_2, u_2); \dots; (t_n, u_n)] \mu =$

si $x = u_1$ **alors** unifier $[(t_2, u_2); \dots; (t_n, u_n)] \mu$

sinon si x apparaît dans u_1 **alors** échec

sinon

soit $\mu_1 = [x \mapsto u_1]$

unifier $(\mu_1(t_2), \mu_1(u_2)); \dots; (\mu_1(t_n), \mu_1(u_n))) (\mu_1 \circ \mu)$

(* Cas terme / variable. *)

unifier $[(t_1, x); \dots; (t_n, u_n)] \mu =$ unifier $[(x, t_1); \dots; (t_n, u_n)] \mu$

(* Cas terme / terme. *)

unifier $[(S(a_1, \dots, a_k), T(b_1, \dots, b_l)); (t_2, u_2); \dots; (t_n, u_n)] \mu =$

si $S = T$ **alors** unifier $[(a_1, b_1); \dots; (a_k, b_k); (t_2, u_2); \dots; (t_n, u_n)] \mu$

sinon échec

Propriétés

L'unificateur de 2 termes (quand il existe) n'est pas unique :

- si μ unifie alors $\forall \theta, \theta \circ \mu$ les unifie aussi
- pour t et u , il existe un « meilleur » unificateur μ , tel que :
 - si θ unifie t et u , alors $\exists \theta'$ t.q. $\theta = \theta' \circ \mu$
 - ▶ i.e. « θ est une extension du meilleur unificateur »
 $(x, B, y) \mathcal{U} (A, B, t) \longrightarrow [x \mapsto A; y \mapsto C; t \mapsto C]$
mais $[x \mapsto A; y \mapsto t]$ plus générale :
 $[x \mapsto A; y \mapsto C; t \mapsto C] = [t \mapsto C] \circ [x \mapsto A; y \mapsto t]$
 - μ est unique, à un renommage près
 - il est appelé **unificateur le plus général**, et on le note quelquefois $\text{mgu}(t, u)$
 - Intérêt pour le **typage** pour obtenir le type le plus général

Alan Robinson, 1965

Voir aussi

Anti-unification ou généralisation (Plotkin 1971, Huet 1976)

Conclusion

Les termes

- représentation de données dans le domaine du calcul symbolique (compilation, logique, *etc.*)
- clos, avec variables
- filtrage et unification : mécanismes de base de «raisonnements mécaniques»

Principes des lang. de progr. INE 213

François Pessaux
(Sur la base d'un cours de Michel Mauny)

ENSTA, édition 2025-2026

`prénom.nom@ensta.fr`



Typage et synthèse de types

- 1 Inférence de types de PCF
- 2 Polymorphisme
- 3 Algorithme

To type or not to type?

1/2

Pas de typage

- grande expressivité des programmes
- programmation « proche de la machine » (OS, drivers)
- risques importants (arrêts après l'erreur)
- assembleur
- C avec (warnings) : typage faible (possibilité de forcer un type)

Typage dynamique

- grande expressivité des programmes
- beaucoup de tests de type (à chaque pas d'exécution, $\Rightarrow$ ↘ efficacité, représentation des données plus volumineuse)
- Lisp, Scheme, Perl, Python, etc.

To type or not to type?

2/2

Typage statique

- moins d'expressivité, quoique ...
- factorisation des tests (éliminés car statique)
- ML, OCaml, C++, Java, Haskell

Inférence vs. vérification

- écrire les programmes sans de préoccuper des types
- plus facile pour le programmeur. . .
... encore que, écrire les types, c'est spécifier un peu
- plus facile (pour le concepteur de langage) de vérifier que d'inférer

PCF, typage statique

Rappel : syntaxe

$$\begin{aligned}
 e ::= & \quad n \in \mathbb{N} \mid b \in \text{bool} \mid x \mid e_1 e_2 \mid \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \\
 & \mid e_1 + e_2 \mid e_1 = e_2 \\
 & \mid \text{let } x = e_1 \text{ in } e_2 \\
 & \mid \text{let rec } f(x) = e_1 \text{ in } e_2 \\
 & \mid \text{fun } x \rightarrow e
 \end{aligned}$$

Éviter les tests dynamiques

- n'ajouter / ne vérifier l'égalité que des entiers
- ne tester que des valeurs booléennes
- n'appliquer que des fonctions

PCF, typage statique

Exemple de problème

- if *booléen* then *entier* else *fonction*
- cette expression : peut-on l'appliquer, l'additionner ?
- la conditionnelle ne peut rendre *qu'un* type
- « *entier* ou *fonction* » n'est pas un type
- on force les deux types de retour à être identiques

⇒ élimine certains programmes qui, pourtant, peuvent avoir un sens

```
def weird (x) :
  if x : return 1
  else : return (lambda y: y + "!")
```

```
print ((weird (True)) + 5)
print ((weird (False)) ("Foo"))
```

Les types de PCF

De quoi avons-nous besoin ?

$\tau ::= \text{int} \mid \text{bool} \mid \tau_1 \rightarrow \tau_2$

Calculer les types

- let rec fact ^{$\text{int} \rightarrow \text{int}$} n ^{$\text{int}$} =
 (if (n ^{int} = ^{$\text{int} \rightarrow \text{int} \rightarrow \text{bool}$} 0 ^{$\text{int}$}) ^{$\text{bool}$} then
 1 ^{int}
 else
 (n ^{int} * ^{$\text{int} \rightarrow \text{int} \rightarrow \text{int}$} (fact ^{$\text{int} \rightarrow \text{int}$} (n ^{$\text{int}$} - ^{$\text{int} \rightarrow \text{int} \rightarrow \text{int}$} 1 ^{$\text{int}$}) ^{$\text{int}$}) ^{$\text{int}$}) ^{$\text{int}$}) ^{$\text{int}$}) ^{$\text{int}$}
- permet de les vérifier aussi

vérifier = calculer, puis comparer
- sens de « comparer » : égalité, compatibilité?

comparer = exiger, forcer l'égalité

Les différents cas

1/2

$n \in \mathbb{N}$: du type int

$b \in \text{bool}$: du type bool

if e_1 **then** e_2 **else** e_3

le type de e_1 doit être bool ; e_2 et e_3 doivent être compatibles

$e_1 + e_2$

arguments entiers, résultat entier

$e_1 = e_2$

types de e_1 et e_2 doivent être compatibles, résultat booléen

let $x = e_1$ **in** e_2

calculer le type de e_1 et l'associer à x durant le typage de e_2

⇒ nécessité d'un **environnement de typage** Γ

Les différents cas

2/2

$x : \Gamma(x)$

$(\text{fun } x \rightarrow e) : \tau_1 \rightarrow \tau_2$

calculer τ_2 pour e en supposant τ_1 pour x
pas de magie, on verra plus loin...

$(e_1 e_2) :$

calcul de $(\tau_2 \rightarrow \tau)$ pour e_1 ; calcul de τ'_2 pour e_2 ; identifier τ_2
et τ'_2

$\text{let rec } f(x) = e_1 \text{ in } e_2$

- calculer le type $(\tau_f \rightarrow \tau_x \rightarrow \tau_{e_1})$ de $(\text{fun } f \rightarrow (\text{fun } x \rightarrow e_1))$
en forçant l'égalité entre τ_f et $\tau_x \rightarrow \tau_{e_1}$;
- puis calculer le type de e_2 en supposant $f : \tau_x \rightarrow \tau_{e_1}$

Système de types

1/2

Types

$\tau ::= \text{int} \mid \text{bool} \mid \tau_1 \rightarrow \tau_2$

Règles

$$\Gamma \vdash n : \text{int} \quad \Gamma \vdash b : \text{bool} \quad \frac{x \in \text{dom}(\Gamma) \quad \Gamma(x) = \tau}{\Gamma \vdash x : \tau}$$

$$\frac{\Gamma \vdash e_1 : \text{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash (\text{if } e_1 \text{ then } e_2 \text{ else } e_3) : \tau}$$

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash (e_1 + e_2) : \text{int}}$$

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash (e_1 = e_2) : \text{bool}}$$

Système de types

2/2

Règles (suite)

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \oplus [x : \tau_1] \vdash e_2 : \tau_2}{\Gamma \vdash (\text{let } x = e_1 \text{ in } e_2) : \tau_2}$$

$$\frac{\Gamma \oplus [x : \tau_1] \vdash e : \tau_2}{\Gamma \vdash (\text{fun } x \rightarrow e) : \tau_1 \rightarrow \tau_2}$$

$$\frac{\Gamma \vdash e_1 : \tau_2 \rightarrow \tau \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1 e_2) : \tau}$$

$$\frac{\Gamma \oplus [f : \tau \rightarrow \tau_1; x : \tau] \vdash e_1 : \tau_1 \quad \Gamma \oplus [f : \tau \rightarrow \tau_1] \vdash e_2 : \tau_2}{\Gamma \vdash (\text{let rec } f(x) = e_1 \text{ in } e_2) : \tau_2}$$

Système de types : remarques

Pas de règles d'erreur

si on peut pas construire d'arbre de dérivation pour un jugement, il n'est pas valide

Une règle par expression

on est proche de l'algorithme de vérification de type
⇒ règles disent comment **vérifier** les types
comment les **deviner** : plus loin dans la séance

Plusieurs occurrences de la même méta-variable (τ_i)

unification pour résoudre ces équations

Polymorphisme à la ML

Jugement pas prouvable

- $\Gamma \vdash (\text{let } f = \text{fun } x \rightarrow x \text{ in } (f f)(1)) : \text{int}$

Pourquoi ?

- $\text{let } f^{\tau \rightarrow \tau} = \text{fun } x^{\tau} \rightarrow x^{\tau} \text{ in } (f^{(i \rightarrow i) \rightarrow (i \rightarrow i)} f^{i \rightarrow i})^{i \rightarrow i}(1^i))$
- $\nexists \tau$ tel que $\tau = i \rightarrow i$ et $\tau = (i \rightarrow i) \rightarrow (i \rightarrow i)$
 f demande à avoir deux types incompatibles, **non unifiables**

Solution : schémas de types

- $f : \forall \alpha. (\alpha \rightarrow \alpha)$
- **instanciable** en les 2 types requis par substitution de α
- polymorphisme **paramétrique**

Polymorphisme à la ML

Types et schémas

$$\begin{aligned}\tau &::= \alpha \mid \text{int} \mid \text{bool} \mid \tau_1 \rightarrow \tau_2 \\ \sigma &::= \forall \vec{\alpha}. \tau\end{aligned}$$

- schéma **trivial** : quand $\vec{\alpha} = \emptyset$
- permet de plonger un type **monomorphe** dans les schémas

Instanciation générique (passage schéma à type)

$\tau_2 \leq \forall \vec{\alpha}. \tau_1$, s'il existe une substitution θ dont le domaine est $\vec{\alpha}$ et telle que $\tau_2 = \theta(\tau_1)$

Généralisation (passage type à schéma)

$\text{gen}(\tau, \Gamma) = \forall \vec{\alpha}. \tau$ où $\vec{\alpha} = \text{vars}(\tau) \setminus \text{fv}(\Gamma)$

correspond à associer un ensemble de solutions à un sous-système d'équations

Polymorphisme à la ML

Règles

[règles grisées : inchangées par rapport au système monomorphe]

$$\Gamma \vdash n : \text{int} \quad \Gamma \vdash b : \text{bool} \quad \frac{x \in \text{dom}(\Gamma) \quad \tau \leq \Gamma(x)}{\Gamma \vdash x : \tau}$$

$$\frac{\Gamma \vdash e_1 : \text{bool} \quad \Gamma \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash (\text{if } e_1 \text{ then } e_2 \text{ else } e_3) : \tau}$$

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash (e_1 + e_2) : \text{int}}$$

$$\frac{\Gamma \vdash e_1 : \text{int} \quad \Gamma \vdash e_2 : \text{int}}{\Gamma \vdash (e_1 = e_2) : \text{bool}}$$

Polymorphisme à la ML

$$\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \oplus [x : \text{gen}(\tau_1, \Gamma)] \vdash e_2 : \tau_2}{\Gamma \vdash (\text{let } x = e_1 \text{ in } e_2) : \tau_2}$$

$$\frac{\Gamma \oplus [x : \forall \emptyset. \tau_1] \vdash e : \tau_2}{\Gamma \vdash (\text{fun } x \rightarrow e) : \tau_1 \rightarrow \tau_2}$$

$$\frac{\Gamma \vdash e_1 : \tau_2 \rightarrow \tau \quad \Gamma \vdash e_2 : \tau_2}{\Gamma \vdash (e_1 e_2) : \tau}$$

$$\frac{\Gamma \oplus [f : \forall \emptyset. \tau \rightarrow \tau_1; x : \tau] \vdash e_1 : \tau_1 \quad \Gamma \oplus [f : \text{gen}(\tau \rightarrow \tau_1, \Gamma)] \vdash e_2 : \tau_2}{\Gamma \vdash (\text{let rec } f(x) = e_1 \text{ in } e_2) : \tau_2}$$

Propriétés

Correction

Soit $\Gamma \vdash e : \tau$, et $\rho : \Gamma$.

Si $\rho \vdash e \Rightarrow r$, alors r est une **valeur** v et son type est τ

Corollaire

Un programme typable s'exécute sans erreur de type.

"Well-typed programs do not go wrong"

Algorithme d'inférence

1/2

But : « deviner » les types (pas juste les vérifier)

Pour e et Γ , produit un type τ et une substitution θ tels que :

$$\theta(\Gamma) \vdash e : \tau$$

```
let rec infer  $\Gamma$  e = match e with
| « n » → (int, [ ])
| « x » → (instance( $\Gamma(x)$ ), [ ])
| « e1 e2 » →
    let ( $\tau_1$ ,  $\theta_1$ ) = infer  $\Gamma$  e1 in
    let ( $\tau_2$ ,  $\theta_2$ ) = infer ( $\theta_1(\Gamma)$ ) e2 in
    let  $\tau_3$  = nouvelle variable in
    let  $\theta_3$  = unif ( $\theta_2(\tau_1)$ ) ( $\tau_2 \rightarrow \tau_3$ ) in
    ( $\theta_3(\tau_3)$ ,  $\theta_3 \circ \theta_2 \circ \theta_1$ )
```

...

Algorithme d'inférence

2/2

```

let rec infer  $\Gamma$  e = match e with
...
| « (fun x  $\rightarrow$  e) »  $\rightarrow$ 
    let  $\tau_0$  = nouvelle variable in
    let  $(\tau_1, \theta_1) = \text{infer } (\Gamma \oplus [x : \tau_0]) \text{ e}$  in
     $((\theta_1(\tau_0) \rightarrow \tau_1), \theta_1)$ 
| « (let x = e1 in e2) »  $\rightarrow$ 
    let  $(\tau_1, \theta_1) = \text{infer } \Gamma \text{ e}_1$  in
    let  $(\tau_2, \theta_2) = \text{infer } (\theta_1(\Gamma) \oplus [x : \text{gen}(\tau_1, \theta_1(\Gamma))]) \text{ e}_2$  in
     $(\tau_2, \theta_2 \circ \theta_1)$ 
| ...

```

Propriétés de l'algorithme

Correction

si $\text{infer } \Gamma \ e = (\tau, \theta)$, alors $\theta(\Gamma) \vdash e : \tau$

Complétude

pour un programme typable, l'algorithme va calculer son type
le plus général, c'est-à-dire celui dont tous ses autres types
sont des instances

Conclusion

Typage statique

- échange de la sécurité contre de la flexibilité
- tenir compte du typage dans l'organisation du code
- éviter la génération de (certains) tests dynamiques

Inférence de types

- évite de spécifier les types des données manipulées
- propage l'information de typage

Propriétés

- correction du typage vis-à-vis de la sémantique d'exécution
- correction et complétude de l'algorithme

Conclusion²

Nous avons vu

- La définition des aspects syntaxique et sémantique des langages de programmation
- Les traits les plus importants des langages classiques (blocs, variables, fonctions/procédures, instructions, structures de données)
- Une mise en œuvre possible de ces traits au travers du langage PCF
- Quelques aspects du calcul symbolique par le biais du langage OCaml

Principes des lang. de progr. INE 213

François Pessaux
(Sur la base d'un cours de Michel Mauny)

ENSTA, édition 2025-2026

`prénom.nom@ensta.fr`



Enregistrements, objets

- 1 Enregistrements
- 2 Objets
- 3 Les objets d'OCaml

Pourquoi des enregistrements ?

Agglomérer des données

- couples, n -uplets
- enregistrements : n -uplets à **composantes nommées**

Exemple

- coordonnées GPS de l'ENSTA :
 - latitude=48,834279
 - longitude=2,283596
- classique, mais peu pratique :
`let ensta = (48.834279, 2.283596)`
- mieux :
`let ensta = { lat=48.834279; long=2.283596 }`
- `ensta.long` au lieu de `snd (ensta)`
⇒ meilleures **lisibilité, maintenabilité**

Enregistrements en OCaml

Recopie partielle

```
type position = { lat : float; long : float; alt : float }  
let monte pos =  
  { lat = pos.lat; long = pos.long; alt = pos.alt +. 1.0 }
```

OCaml permet d'abrégier en

```
let monte pos = { pos with alt = pos.alt +. 1.0 }
```

Généralisation

```
let aller_a pos la lo = { pos with lat = la; long = lo }
```

Attention : copie et non partage comme l'induit

```
let ensta = { lat=48.834279; long=2.283596 }  
let ensta' = ensta
```

Enregistrements en PCF : syntaxe

Syntaxe

$$\begin{aligned} e ::= & n \in \mathbb{N} \mid b \in \text{bool} \mid x \mid e_1 e_2 \mid \text{if } e_1 \text{ then } e_2 \text{ else } e_3 \\ & \mid e_1 + e_2 \mid e_1 = e_2 \mid \text{fun } x \rightarrow e \\ & \mid \text{let } x = e_1 \text{ in } e_2 \mid \text{let rec } f(x) = e_1 \text{ in } e_2 \\ & \mid \{\ell_1 = e_1; \dots; \ell_n = e_n\} \\ & \mid e.\ell \\ & \mid \{e \text{ with } \ell = e'\} \end{aligned}$$

Enregistrements en PCF : sémantique opérationnelle

Valeurs

$$v ::= c \mid \text{Valf}(x, e, \rho) \mid \text{Valfr}(f, x, e, \rho) \\ \mid \text{Record}((\ell_1, v_1), \dots, (\ell_n, v_n))$$

Règles d'évaluation

$$\frac{\rho \vdash e_1 \Rightarrow v_1 \quad \dots \quad \rho \vdash e_n \Rightarrow v_n}{\rho \vdash \{\ell_1 = e_1; \dots; \ell_n = e_n\} \Rightarrow \text{Record}((\ell_1, v_1), \dots, (\ell_n, v_n))} (\text{Rec})$$

$$\frac{\rho \vdash e \Rightarrow \text{Record}(\dots, (\ell, v), \dots)}{\rho \vdash e.\ell \Rightarrow v} (\text{RAcc})$$

$$\frac{\rho \vdash e \Rightarrow \text{Record}((\ell_1, v_1), \dots, (\ell, v), \dots, (\ell_n, v_n)) \quad \rho \vdash e' \Rightarrow v'}{\rho \vdash \{e \text{ with } \ell = e'\} \Rightarrow \text{Record}((\ell_1, v_1), \dots, (\ell, v'), \dots, (\ell_n, v_n))}$$

Enregistrements en PCF : sémantique opérationnelle

Généralisation

On écrit

$$\{e \text{ with } \ell_1 = e_1; \dots; \ell_n = e_n\}$$

pour

$$\{\dots \{e \text{ with } \ell_1 = e_1\} \dots \text{ with } \ell_n = e_n\}$$

C'est du « sucre syntaxique »

- càd une écriture simplifiée d'une combinaison de constructions existantes
- la sémantique de la forme abrégée (le « sucre ») = sémantique de la forme complète
- exemple : la conjonction booléenne ($e_1 \ \&\& \ e_2$) est du « sucre » pour (**if** e_1 **then** e_2 **else** **false**)
- idem pour la disjonction

Objets

Exemple

- un point (2D) est donné par :
 - son abscisse x
 - son ordonnée y
 - une méthode de déplacement, qui modifie les coordonnées

Objets représentés par des enregistrements

- les **variables d'instance** sont des champs contenant des **références** (ou des champs mutables)
- les **méthodes** sont des champs contenant des **fonctions**

Objets

Représentation d'un objet point : 1er essai

```
let p = {  
  x = ref 3 ;  
  y = ref 4 ;  
  move = fun dx dy →  
    x := !x + dx ;  
    y := !y + dy  
}
```

⇒ paramétrer les méthodes par l'objet courant

Problème

- x et y ne sont **pas connus** de move
- si move accède directement aux champs x et y de p, alors son code **ne peut être partagé** entre différents objets

Objets

Représentation d'un objet point

```
type ty_p = { x: int ref; y: int ref; move: ty_p → ... }  
let p = {  
  x = ref 3 ;  
  y = ref 4 ;  
  move = fun this dx dy →  
    this.x := !(this.x) + dx ;  
    this.y := !(this.y) + dy }
```

Autre syntaxe

```
let p = {  
  x = ref 3 ;  
  y = ref 4 ;  
  move this dx dy =  
    this.x := !(this.x) + dx ;  
    this.y := !(this.y) + dy }
```

Appel de méthode

`p.move p 1 1`

Autre notation

On écrit $e\#m$ pour

`let this = e in this.m(this)`

`p#move 1 1`

Polymorphisme

Appeler une méthode provenant d'objets différents

```
let p1 = { x = ref 4; y = ref 2; move = ... }  
let p2 = { x = ref 1; y = ref 3; move = ...; c = ref "rouge" }  
let do_move p dx dy = p#move dx dy  
do_move p1 1 0 ; do_move p2 0 1 ;
```

Difficultés d'implémentation : *dynamic dispatch*

- types de p_1 et p_2 **incompatibles**
⇒ modifier le typage (*enregistrements extensibles, rangées*)
- où est la méthode `move` ?
 - dans l'objet ? À quelle position ? ⇒ hachage et recherche dynamique
 - dans la classe ? Accès dynamique à la classe.

Solutions

- OCaml, JavaScript : les méthodes sont dans l'objet
- C++ : l'objet dispose d'un pointeur vers la table des méthodes de sa classe (*virtual table*)

Classes

Les classes sont des générateurs d'objets

```
let class_point () =  
  { x = ref 3; y = ref 4; move = fun this → ... }  
  
let p = class_point()
```

Notation

On écrira **new c**
pour **c()**

Exemple

```
let p1 = new class_point in  
let p2 = new class_point in  
p1#move 150 100
```

Héritage

Une sous-classe réutilise la super-classe

```
let class2Dpoint (xinit, yinit) =  
  { x = ref xinit ; y = ref yinit ;  
    move this dx dy = ... ;  
    reset this = (this.x := 0; this.y := 0) }  
  
let class3Dpoint (xinit, yinit, zinit) =  
  let super = new class2Dpoint (xinit, yinit) in  
  { x = super.x ; y = super.y ; z = ref zinit ;  
    move this dx dy dz =  
      (super.move this dx dy ; this.z := !this.z + dz) ;  
    reset this = (super.reset this ; this.z := 0) }
```

- Variables d'instance **références** $\Rightarrow$ **partage**.

Héritage et *dynamic dispatch*

Quelle méthode est appelée ?

```
let l = [new class2Dpoint ; new class3Dpoint]  
let pt_reset point = point#reset  
List.iter pt_reset l
```

- reset 2D ou reset 3D ?
- Dépend du point effectif.
- Généralement inconnu à la compilation.
- Méthode sélectionnée **dynamiquement**.

Les objets d'OCaml

Fonctionnalités disponibles

- classes,
- typage et inférence de type statiques,
- les objets portent (des pointeurs vers) leurs méthodes,
- héritage multiple.

OCaml : classes, objets et méthodes

```
class point =  
  object  
    val mutable x = 0  
    method get_x = x  
    method move d = x ← x + d  
  end;;  
class point :  
  object val mutable x : int method get_x : int  
    method move : int → unit end  
  
let p = new point;;  
val p : point = <obj>  
(* point est une abréviation pour  
   <get_x : int; move : int → unit> *)
```

OCaml : appels de méthodes

```
p#get_x ;;
```

```
- : int = 0
```

```
p#move 3 ;;
```

```
- : unit = ()
```

```
p#get_x ;;
```

```
- : int = 3
```

OCaml : classes paramétrées

```
class point xinit =  
  object  
    val mutable x = xinit  
    method get_x = x  
    method get_offset = x - xinit  
    method move d = x ← x + d  
  end ;;  
class point : int →  
  object  
    val mutable x : int  
    method get_offset : int  
    method get_x : int  
    method move : int → unit  
  end
```


OCaml : classes paramétrées

this est un paramètre explicite

```
class printable_point xinit =  
  object (this) (* On nomme l'objet (this, self, moi, ...) *)  
    val mutable x = xinit  
    method get_x = x  
    method move d = x ← x + d  
    method print = print_int this#get_x  
  end ;;
```

OCaml : héritage

```
class colored_point x (c : string) =  
  object  
    inherit point x  
    val c = c  
    method color = c  
  end ;;
```

Les méthodes peuvent être virtuelles, privées, etc.

OCaml : polymorphisme

```
let get_x o = o#get_x
val get_x : < get_x :  $\alpha$ ; .. >  $\rightarrow$   $\alpha$  = <fun>
```

```
let p = new point 3
and q = new colored_point 4 "blue" ;;
val p : point = <obj>
val q : colored_point = <obj>
```

```
let l = [p; q] ;;
Error: This expression has type colored_point
      but an expression was expected of type point
      The second object type has no method color
```

(* Contrainte de type : force à être du type parent. *)

```
let colored_point_to_point cp = ((cp : colored_point) :> point) ;;
val colored_point_to_point : colored_point  $\rightarrow$  point = <fun>
```

```
let l = [ p; (colored_point_to_point q) ] ;;
val l : point list = [<obj>; <obj>]
```

```
List.map get_x l ;;
- : int list = [3; 4]
```

Conclusion

- Les enregistrements peuvent servir de base à l'encodage des objets.
- Certains aspects « dynamiques » des objets sont assez difficiles à compiler efficacement.
- Le typage (pas abordé ici) et la synthèse de types sont complexes.
- On peut assez facilement réaliser un interprète pour PCF (+ enregistrements) + objets.

Principes des lang. de progr. INE 213

François Pessaux
(Sur la base d'un cours de Michel Mauny)

ENSTA, édition 2025-2026

prénom.nom@ensta.fr



Gestion et récupération automatique de mémoire

Glanage de C
Garbage ellules
 ollection

1 La mémoire

2 Recyclage

- GC à balayage et copiants
- Comptage de références
- GC copiants à générations

Allocation dynamique de mémoire

Allocation dynamique de mémoire

- stocker les variables, structures, appels de fonction, *etc.*

Pour la plupart des langages la taille totale de l'espace mémoire nécessaire au stockage de ces objets ne peut pas être déterminée au moment de la compilation... sauf pour Fortran.

Certains traits de langages nécessitent une allocation dynamique :

- la récursion ;
- les pointeurs et l'allocation explicite (**new**, malloc, cons) ;
- les structures de données dynamiques (listes, arbres, tableaux dynamiques) ;
- les fonctions comme « valeurs » (création de fermetures).

Différents types de mémoire

Allocation statique

Allouée statiquement pour stocker les données dont la taille ne change pas durant l'exécution du programme :

- variables globales
- chaînes littérales
- tableaux (globaux) dont la taille est connue à la compilation
- ...

Allouée dès le début de l'exécution du programme, et non recyclable.

La mémoire allouée dynamiquement sert à allouer des données dont la taille totale **n'est pas prévisible à la compilation**.

Différents types de mémoire

Alloc. dynamique à durée de vie prévisible

La pile contient les adresses de retour, les variables locales, les arguments de la fonction courante.

Allocation sur la pile

- *LIFO* ou *last in, first out* : ajuste les durées de vie aux blocs d'activation de fonction ;
- bloc d'activation : retiré de la pile lorsqu'un appel de fonction se termine
 - ⇒ désallocation automatique des variables locales lorsqu'on sort de leur portée.

La pile : pour quelles données ?

- constantes (entières, booléennes), les structures C, etc.
- données avec durée de vie n'excédant pas l'appel de fonction courant
- ...et dont la taille est modeste

Différents types de mémoire

Données à durée de vie imprévisible

Les **pointeurs** référencent une donnée

- la valeur d'une variable de type pointeur est l'adresse de l'objet pointé
- sont traités comme des entiers
- permettent les **alias** : plusieurs pointeurs vers la même adresse
- supprimer un pointeur $\nrightarrow$ donnée référencée devient **inutile** !

Durée de vie des objets

- une fonction peut retourner un pointeur vers
 - une donnée non locale à la fonction
 - une zone allouée devant survivre à l'appel de la fonction
- durée de vie **imprévisible**

La mémoire allouée dynamiquement

Pointeurs

```
struct record {  
    ...  
};  
  
record* f () {  
    record *ptr ;  
    ptr = new record ;  
    ... ;  
    return ptr ;  
}
```

```
void g () {  
    record *ptr ;  
    ptr = new record ;  
    ... ;  
    b = ptr ;  
    return ;  
}
```

La variable contenant le pointeur est désallouée, mais **pas ce qui est pointé** !

La mémoire allouée dynamiquement

Les pointeurs

On ne peut donc pas tout désallouer au retour des appels de fonctions :

- certaines données **survivent** à ces appels : leur durée de vie est **supérieure** à celle de la fonction qui les a créées
- ⇒ **ne peuvent pas** être représentées dans la **pile**

Allocation

- dans le **tas** : une zone mémoire dédiée
- 1 allocation ⇒ 1 **bloc de mémoire contiguë**.

La mémoire allouée dynamiquement

Et quand le tas est plein ?

- certains blocs alloués peuvent ne plus être nécessaires
- l'espace mémoire qu'ils occupent peut être **recyclé**

Récupération

- explicite, « manuelle » (`free`, `delete`)
- implicite, automatique : GC (*garbage collection*, glanage de cellules)

Recyclage explicite

Gestion manuelle

```
struct cell {  
    int val ;  
    struct cell *suiv ;  
} ;  
typedef cell *liste ;
```

```
liste cons (int v, liste vs) {  
    liste nouv = new (cell) ;  
    nouv->val = v ;  
    nouv->suiv = vs ;  
    return nouv ;  
}  
  
void libereArbre (arbre a) {  
    if (a ≠ NULL) {  
        libereArbre (a->gauche) ;  
        libereArbre (a->droit) ;  
        delete a ;  
    }  
}
```

Recyclage implicite

Gestion automatique

```
type intlst =  
  Null  
| Cell of int * intlst
```

```
let cons v vs =  
  Cell (v, vs)
```

```
let rec append vs1 vs2 =  
  match vs1 with  
  | Null → vs2  
  | Cell (n, ns) → cons n (append ns vs2)
```

```
let vs1 = cons 0 (cons 1 (cons 2 Null)) in  
append vs1 (cons 3 (cons 4 null))
```

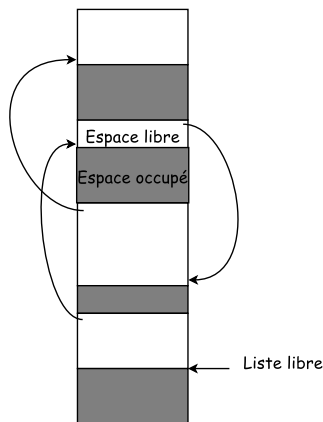
```
let vs1 = [0; 1; 2] in vs1 @ [3; 4]
```

La liste vs1 est devenue inutile, car elle n'est plus référencée.

Organisation du tas

La liste libre

- Les blocs de mémoire libre sont chaînés entre eux, en une liste appelée **liste libre** (*free list*).
- Allocation : quand un bloc est requis, il est **pris** dans (et **retiré** de) la liste libre.
- Libération/recyclage : quand un bloc est récupéré, il est **inséré** dans la liste libre.



Organisation du tas : la liste libre

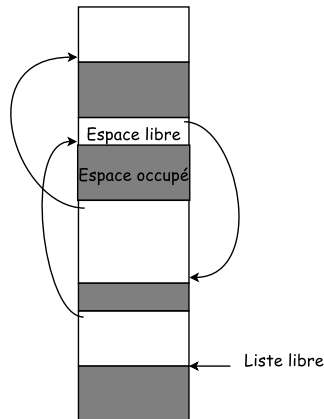
Récupération efficace mais ...

Désavantages

- Perte de localité des données
- Parcours de la liste libre pour trouver un bloc suffisamment grand
- Compaction des blocs libres requise

En bref

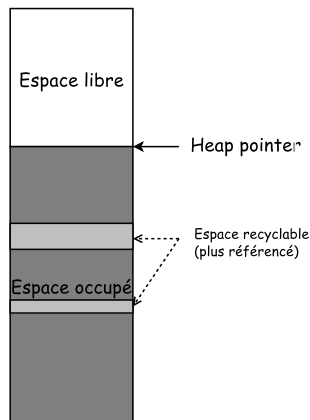
- Recyclage efficace.
- Allocation potentiellement coûteuse.



Organisation du tas

La méthode du «pointeur de tas»

- L'espace disponible est **un bloc contigu** de mémoire.
- Un **pointeur de tas**, (*heap pointer*) analogue à un pointeur de pile, désigne le prochain mot-mémoire **libre**.
- Allocation : modification de pointeur de tas.



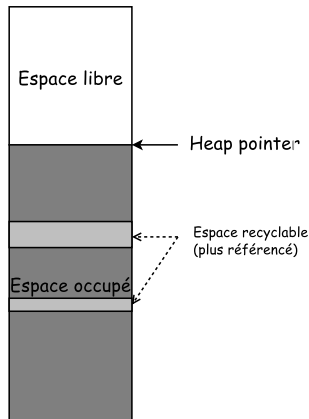
Organisation du tas

Allocation efficace mais ...

Comment recycler ?

- **Compacter** les blocs « vivants », c'est-à-dire les « pousser » jusqu'à ce qu'ils deviennent **adjacents**.
- Comment faire, sans que cela soit trop coûteux ?

Réponse un peu plus loin...



Recyclage mémoire

C, Pascal, Ada n'utilisent pas intensivement le tas.

- (généralement) pas de gestion auto. de la mémoire.
- recyclage mémoire **explicite**, par le programme(ur).
- free, delete.

Langages de haut niveau

- OCaml, Prolog, Java, C#, ...
- Javascript, Python, ...
- utilisent **intensivement** le tas
- disposent d'un **gestionnaire automatique de mémoire** (GC)

Utilisation intensive du tas :

- OCaml, JavaScript, Python, ... : listes, tableaux, chaînes de caractères, structures de données, fermetures (représentations de valeurs fonctionnelles), objets, etc.
- Java, C# : objets, tableaux, etc.
- ...

Catégories de GC

En résumé, les tâches assurées par un GC sont les suivantes :

- ➊ Identifier les blocs alloués qui ne pourront plus être référencés pendant l'exécution : les **blocs morts**, et
- ➋ **recycler** la mémoire correspondante, afin qu'elle puisse être réutilisée.

Efficacité d'un GC

- **coût temporel** rajouté à l'exécution
- **espace mémoire** requis
- **capacité à récupérer** la mémoire

Catégories de GC

Identification des blocs morts

- **Les GC à balayage et les GC copiants**
 - parcourent la mémoire lorsque le tas est plein pour **identifier les blocs vivants**
 - considèrent tous les autres blocs comme **morts**, et **récupèrent** la mémoire qu'ils occupent.
- **Les GC à compteurs de références**
 - sont capables de déterminer à tout moment de l'exécution si un bloc devient mort
 - et récupèrent **à ce moment** la mémoire occupée par ce bloc.

Les GC à balayage et copiants

Durant l'exécution les blocs alloués sur le tas peuvent être référencés de plusieurs façons : comme valeur de variables, composants de structure, *etc.*

En particulier, un bloc x **vivant** (c'est-à-dire est encore référencé) si :

- ❶ il est pointé directement par une **variable**
- ❷ il y a un **registre** qui contient une valeur temporaire ou intermédiaire qui pointe sur x
- ❸ il existe un bloc y dans le tas qui est **vivant** et qui contient un **pointeur** vers x .

Les GC à balayage et copiants

Tous les blocs vivants sont accessibles par un parcours de graphe

- les points de départ de ce parcours sont les variables locales sur la pile, les variables globales, et les registres ce sont les racines du parcours
- tout bloc non accessible par ce parcours est mort et peut être recyclé.
- **Attention** : on doit
 - savoir distinguer les pointeurs des autres données !
 - connaître la taille des blocs

Les GC copiants et à balayage effectuent un parcours du graphe mémoire :

ils diffèrent par les actions qu'ils effectuent pendant et après le parcours.

Les GC à balayage

Idée de base

- chaque bloc contient un **bit de marque**
- la mémoire libre est dans une **liste libre**
- quand le tas se remplit, l'exécution est **suspendue** et le processus de récupération de mémoire commence.

Phase de marquage

- le GC **parcourt le tas**, à partir des **racines**, et positionne le *bit de marque de chaque bloc qu'il rencontre* ;
- *chaque bloc qui reste non marqué est inaccessible.*

Phase de balayage

- le GC parcourt ensuite **toute la mémoire**, plaçant chaque bloc **non marqué** dans la liste **libre**.

Les GC à balayage

```
collecte_par_marquage_balayage () {  
    pour chaque racine  $r$  :  
        marquer ( $r$ ) ;  
    balayer () ;  
}  
  
marquer ( $p$ ) {  
    si le bloc pointé par  $p$  n'est pas marqué, alors :  
        on marque le bloc pointé par  $p$  ;  
    pour chaque champ  $r$  de ce bloc :  
        si  $r$  est un pointeur alors :  
            marquer ( $r$ ) ;  
}  
  
balayer () {  
    pour chaque bloc mémoire  $b$  du tas :  
        si  $b$  est non marqué, alors :  
            insérer ( $b$ , freelist) ;  
}
```

Coût du GC à balayage

Si L = taille de la mémoire «vivante»
et M = taille du tas
alors

$$\begin{aligned}\text{Coût de GC}_{\text{balayage}} &= O(\text{marquage}) + O(\text{balayage}) \\ &= O(L) + O(M) \\ &= O(M), \text{ puisque } M > L\end{aligned}$$

Coût du GC à balayage : proportionnel à la taille du tas,
indépendamment du volume de blocs vivants.

Désavantages des GC à balayage

- ne compactent pas (liste libre)
⇒ allocation coûteuse
- l'exécution s'arrête durant le GC
 - mal adaptés aux programmes temps-réel
 - appelés *stop-and-collect*

Les GC copiants

Idée de base

- il y a 2 tas, dont un seul est utilisé à la fois
 - le tas FROM est utilisé
 - l'autre s'appelle le tas TO
- les blocs sont alloués dans le tas FROM
- quand FROM est plein, le GC commence

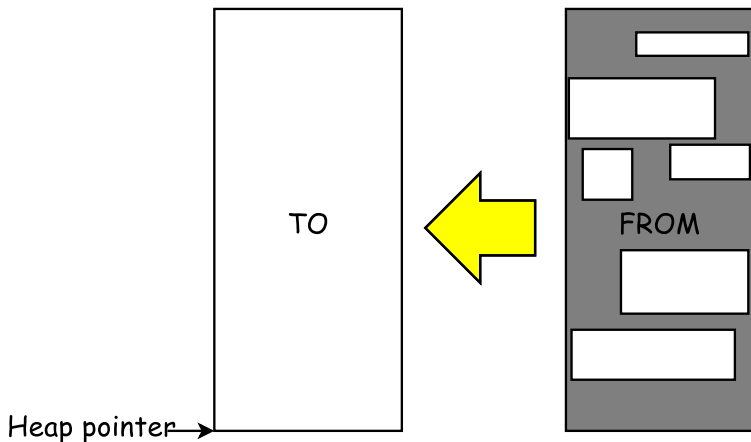
Phase de copie (de FROM vers TO)

- le GC parcourt le tas FROM, à partir des racines, et recopie les blocs qu'il rencontre dans le tas TO (initialement vide)
- à la fin, TO contient tous les blocs vivants

Phase d'échange

- le GC échange FROM et TO, et l'exécution reprend.

Les GC copiants



Les GC copiants : propriétés

Chaque bloc vivant est copié dans le premier espace disponible du tas TO :

- à la fin de la recopie, la mémoire libre aura été **recompactée** (un seul bloc libre)

Puisque les blocs ont été déplacés de FROM vers TO, les pointeurs contenus dans les blocs doivent être **mis à jour** vers leur nouvelle adresse :

- effectué en laissant une « adresse distante » dans l'ancien bloc après son déplacement
- si un pointeur vers l'**ancien** bloc est rencontré plus tard, le pointeur est mis à jour avec l'**adresse distante** du bloc.

Une adresse distante (dans TO) est facile à distinguer d'une autre (dans FROM).

Les GC copiants

```
collecte_par_recopie () {  
  pour chaque pointeur racine  $p$  :  
     $p := \text{transférer}(p)$  ; (* Récupérer la nouvelle racine. *)  
}  
  
transférer ( $p$ ) {  
  si  $p$  pointe vers un bloc contenant une adresse distante  $q$ , alors :  
    renvoyer  $q$  ;  
  sinon :  
    soit  $q = \text{copie}(p, T0)$  ;  
    écrire  $q$  comme adresse distante  
      à la place du bloc pointé par  $p$  ;  
    (* Parcours des champs après copie, donc depuis  $T0$ . *)  
    soit  $flds$  les champs de  $q$  contenant des pointeurs ;  
    pour chaque champ  $qf$  de  $flds$ ,  
       $qf := \text{transférer}(qf)$  ;  
    renvoyer  $q$  ;  
}
```

Coût du GC copiant

Si L = taille de la mémoire «vivante»
et M = taille du tas
alors

$$\text{Coût de GC}_{\text{copiant}} = O(L)$$

À l'inverse des GC à balayage le coût est **proportionnel** au volume des **blocs vivants**, et non pas à la taille du tas.

Avantages

- mémoire **compactée**

Inconvénients

- l'exécution **s'arrête** durant le GC
- nécessite un espace mémoire **double**

Les GC à compteurs de références

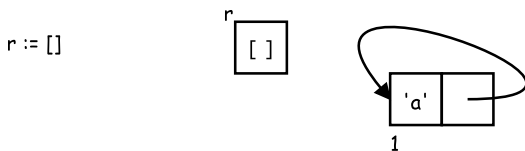
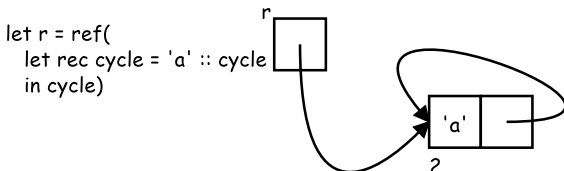
Idée complètement différente

Chaque bloc contient un **champ** appelé **compteur de références**

- contient le nombre de fois que ce bloc est référencé (nombre de pointeurs **vers** ce bloc)
- quand un pointeur vers un bloc est **copié**, le compteur du bloc est **incrémenté**
- quand un pointeur vers un bloc est **supprimé**, le compteur est **décrémenté**
- quand le compteur devient **0**, le bloc est **recupéré**, c'est-à-dire :
 - chaque pointeur qu'il contient est détruit (impliquant des décréments de leur compteur $\Rightarrow$ effet cascade)
 - le bloc est placé dans la **liste libre**, prêt à être recyclé

Les GC à compteurs de références

Les cycles posent un problème



Nécessité d'avoir un GC à balayage comme backup (ou d'accepter les **fuites mémoire**)

Les GC à compteurs de références

Désavantages

- les opérations simples sur les pointeurs deviennent **coûteuses**
 - ▶ changer la valeur d'une variable pointeur :
 - écrire la nouvelle valeur
 - **incrémenter** le compteur du **nouveau** bloc pointé
 - **décrémenter** le compteur de l'**ancien** bloc pointé
 - le **tester**, si 0 **libérer** le bloc (effet cascade possible)
- difficile à implémenter (bugs faciles lors de génération de code créant des temporaires, passage de paramètres, etc.)
- chaque bloc doit avoir un compteur (qui peut déborder)
- GC à balayage « de secours »

Résumé

GC à balayage assez peu efficaces.

GC copiants efficaces, compactants, et profitent du bas prix de la mémoire.

GC à compteurs de références conviennent à des situations très particulières (pas de cycles, pas de risque de débordements, etc.)

GC copiants à générations

Observations expérimentales : dans certains langages

- les blocs **jeunes meurent vite**,
- les blocs qui **survivent vivent vieux**,
- les jeunes pointent vers les vieux, le plus souvent

Raisons :

- de nombreux blocs sont très éphémères : données temporaires, valeurs intermédiaires
- ceux qui « survivent » correspondent à des blocs « importants » (variables globalement accessibles, structures de données centrales au programme, ...)

GC copiants à générations

Les GC copiants à générations exploitent cette propriété :

- avec un nombre de tas ≥ 2
- chaque tas contient des blocs d'âges « similaires »
 - chacun de ces tas est appelé **génération**
 - jeunes générations, (générations intermédiaires), vieilles générations
- examiner les blocs avec **probabilité forte** d'être **morts**

GC copiants à générations

Les jeunes générations sont collectées plus fréquemment que les vieilles :

- une proportion importante des blocs sont morts dans les jeunes générations
- le coût du GC est **réduit** car seulement **une partie** des blocs **vivants** est parcourue à chaque GC (une génération)

Les blocs parcourus (vivants) dans une génération sont **copiés** dans la **génération suivante** (ils vieillissent)

- lorsque la génération suivante est pleine, elle est recyclée à son tour, *etc.*

GC copiants à générations

Les « vieux » blocs ne pointent que **rarement** vers les jeunes

- dans les langages (purement) fonctionnels soumis à une évaluation stricte, **ils ne le peuvent pas**
- les racines de la jeune génération sont *a priori* les registres, variables locales et variables globales **qui pointent vers la jeune génération**

Un bloc d'une vieille génération ne peut pointer vers un « jeune » que par une **affectation**

- ⚠ risque de libérer un bloc jeune pointé seulement par un vieux lors du recyclage des jeunes
- ⇒ une affectation établissant un pointeur d'un vieux vers un jeune déclare le vieux comme *référence entrante* (**racine**)
- **chaque génération** dispose de sa liste de références entrantes

Conclusion

GC à balayage assez peu efficaces.

GC à compteurs de références conviennent à des situations très particulières (pas de cycles, pas de risque de débordements, etc.)

GC copiants efficaces, compactants, et profitent du bas prix de la mémoire.

GC copiants générationnels efficaces, compactants, et profitent du bas prix de la mémoire.

- répartissent la pause du GC en une pause par génération
- pour des langages avec relativement peu d'affectations

Principes des lang. de progr.

INE 213

François Pessaux

ENSTA ParisTech, édition 2020-2021

prénom.nom@ensta.fr



Interprétation abstraite

Par les tests logiciels

- Déroulement :
 - Création des jeux de tests.
 - Exécution.
 - Comparaison à l'attendu.
- Méthode **dynamique**.
- Test **exhaustif** souvent **impossible**.
- Trouve des bugs, n'assure **pas leur absence**.

Par la preuve partielle

- Preuve « partielle » : juste **certaines** propriétés du programme.
- Méthode **statique** (pas d'exécution).
- Exhaustivité : couvre **toutes** les exécutions possibles.
- Besoin d'outils automatiques.
- Vérification automatique de propriétés arbitraires sur des programmes arbitraires :
 - **Indécidable** (théorème de Rice).
 - $\Rightarrow$ Réponse outil parfois « *je ne sais pas* ».
 - $\Rightarrow$ Réponse outil parfois « *bug présent* » alors que non.
- « Preuve » $\Rightarrow$ **pas oublier** d'occurrences de bugs recherchés.

Programme

```
p0
  y = 0
p1
  while x > 0
p2
    x = x / 2
p3
    y = y + x
p4
  exit
p5
```

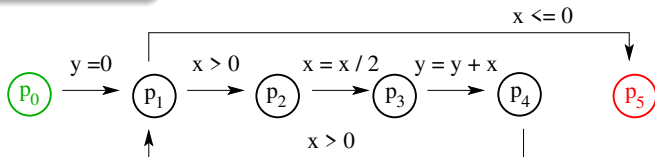
Propriétés : exemples

- $\forall p_i, x \geq 0$ (faux).
- $\forall x$, pas de division par 0 (vrai).
- $\forall x$, pas de débordement arithmétique (vrai – subtil).
- $\forall x, y \geq 0$ à la fin du programme (vrai car pas débordement).

Trace d'exécution

```
p0  y = 0
p1  while x > 0
p2      x = x / 2
p3      y = y + x
p4  exit
p5
```

- **Interprète** : simule l'exécution.
- Travaille sur le **graphe de flot de contrôle** (CFG).
- **État** : valeur de chaque variable à **un** point du programme.
- Mémoriser les états d'une exécution : **trace d'exécution**.



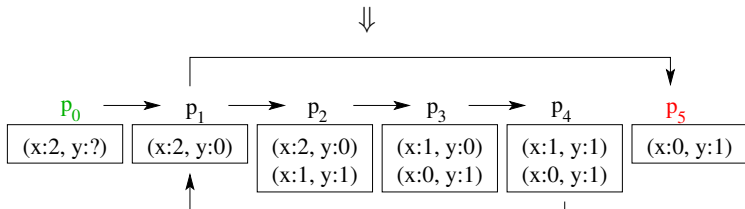
Pour $x = 2$:

$p_0 (x : 2, y : ?) \rightarrow p_1 (x : 2, y : 0) \rightarrow p_2 (x : 2, y : 0) \rightarrow p_3 (x : 1, y : 0)$
 $\rightarrow p_4 (x : 1, y : 1) \rightarrow p_2 (x : 1, y : 1) \rightarrow p_3 (x : 0, y : 1) \rightarrow p_4 (x : 0,$
 $y : 1) \rightarrow p_5 (x : 0, y : 1)$

- Interprète : basé sur la sémantique **opérationnelle** du langage
 - ▶ Sémantique concrète.
- $\Rightarrow$ Une trace peut être **infinie** (programme qui boucle).
- Il peut y avoir une « **infinité** » de traces
 - ▶ Combinaisons de toutes les valeurs possibles de toutes les entrées.
- $\Rightarrow$ Interprète peut donc **ne pas terminer**.
- $\Rightarrow$ Fâcheux pour un outil ☹
- Besoin de faire des **approximations**.

- Remarque : points de programme en nombre **fini**.
- **Regrouper** les états d'un **même** point de programme.
- En cas de boucle :
 - on accumule les états,
 - on recommence tant que nouveaux états ajoutés.
 - $\Rightarrow$ Calcul de **point fixe**.

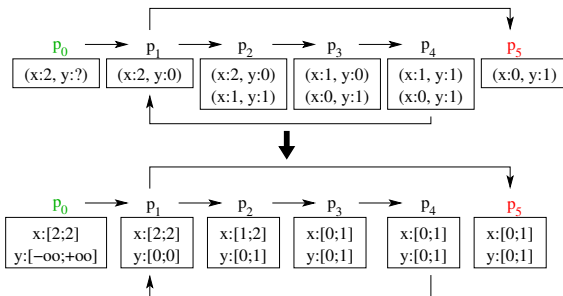
$p_0 (x : 2, y : ?) \rightarrow p_1 (x : 2, y : 0) \rightarrow p_2 (x : 2, y : 0) \rightarrow p_3 (x : 1, y : 0) \rightarrow p_4 (x : 1, y : 1)$
 $\rightarrow p_2 (x : 1, y : 1) \rightarrow p_3 (x : 0, y : 1) \rightarrow p_4 (x : 0, y : 1) \rightarrow p_5 (x : 0, y : 1)$



- $\Rightarrow$ Réduit la **longueur** des traces.
- Mais : nombre d'états potentiellement **très** (trop) **grand**.

Sémantique abstraite

- Pas forcément besoin de conserver toutes les valeurs.
- Regrouper plusieurs valeurs concrètes ensemble.
- Vue « ensembliste » : valeur abstraite.
- Exemple des intervalles : $\{4, 8, 25\} \mapsto [4; 25]$.
- Exemple des signes : $x < 0 \mapsto -, x = 0 \mapsto 0, x > 0 \mapsto +$.



- Règle la terminaison : devient calculable si ... (conditions après).
- Domaines plus complexes et plus précis existent.

Sémantique concrète

- Vérification propriété sur 1 exécution.
- Non calculable.

Sémantique collectrice

- Vérification propriété sur 1 exécution, réduction des traces.
- A priori calculable, mais à un prix déraisonnable.

Sémantique abstraite

- Sur-approxime la sémantique collectrice.
- Calculable.

Remarques

- Dès collectrice, représentation d'exécutions impossibles.
- Concrète $\subseteq$ collectrice $\subseteq$ abstraite.
- Mais il faut respecter des conditions...

Analyse doit terminer

- $\Rightarrow$ Calcul de **point fixe** doit terminer.
- Contraintes sur le **domaine abstrait** ?
- Contraintes sur les **opérations** sur le domaine abstrait ?

Sémantique abstraite

- Garantir la **correction** vis-à-vis de la sémantique concrète.
- Opérations liées aux instructions concrètes $\rightarrow$ opérations abstraites.
- **Liens** entre sémantiques **concrète** et **abstraite** ?

$\Rightarrow$ Plusieurs précautions pour une analyse **correcte**.

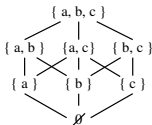
Définition

Un **treillis** est un ensemble partiellement ordonné dans lequel chaque paire d'éléments admet :

- une borne **supérieure** : $\forall x, y \in D, x \sqcup y \in D$,
- une borne **inférieure** : $\forall x, y \in D, x \sqcap y \in D$.

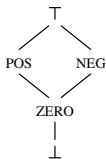
Exemple

Ensemble des parties d'un ensemble muni de l'inclusion.



Notation

- $\top$: borne supérieure.
- $\perp$: borne inférieure.



Notations

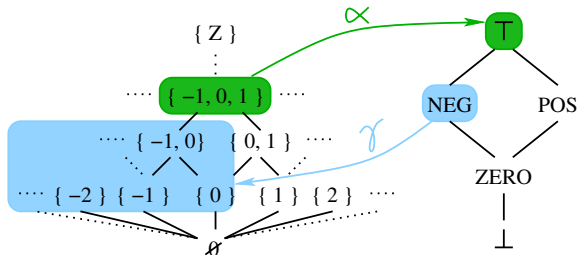
- Ensemble **concret** : D .
- Ensemble **abstrait** : $D^\#$.
- Opération (instruction) **concrète** : f .
- Opération (instruction) **abstraite** : $f^\#$.
- **Domaine** : ensemble muni de ses opérations.

Théorème de Knaster-Tarski *Si T est un treillis complet et $f : T \rightarrow T$ une application croissante, alors le sous-ensemble ordonné des points fixes de f est un treillis complet. D'où f a un plus petit et un plus grand point fixe.*

- $\Rightarrow$ Domaine abstrait doit être un **treillis complet**.
- $\Rightarrow$ Opérations sur le domaine abstrait : **croissantes**.
- Si hauteur du treillis infinie : besoin d'une opération d'**élargissement**.

Définitions

- Une fonction d'**abstraction** α associe à chaque objet concret o un objet abstrait $o^\#$ (simplification de o).
- $\alpha : D \rightarrow D^\#$.
- Une fonction de **concrétisation** γ associe à chaque objet abstrait $o^\#$ le plus grand objet o qu'il approxime.
- $\gamma : D^\# \rightarrow D$.
- α γ pas quelconques : **relient** les domaines abstrait et concret.

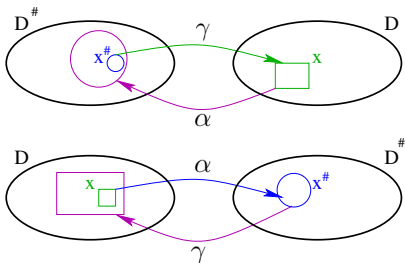


Lien entre α et γ

α et γ doivent former une **correspondance de Galois** entre D et $D^\#$:

- α et γ **croissantes**,
- $\forall x^\# \in D^\#, x^\# \sqsubseteq_\# \alpha(\gamma(x^\#))$,
- $\forall x \in D, \gamma(\alpha(x)) \sqsupseteq x$.

$\Rightarrow$ Préservation de la sûreté, perte de précision.



Notation $(D, \sqsubseteq) \xleftrightarrow[\alpha]{\gamma} (D^\#, \sqsubseteq_\#)$

Correction de la sémantique abstraite

- Sémantique abstraite : **abstraction** des opérations **concrètes**.
- $f \rightsquigarrow f^\#$.
- $f^\#$ **correcte** si $\gamma(f^\#(x^\#)) \sqsupseteq f(\gamma(x^\#))$

Lemme de correction de l'analyse

Soit $(D, \sqsubseteq) \xleftrightarrow[\alpha]{\gamma} (D^\#, \sqsubseteq_\#)$.

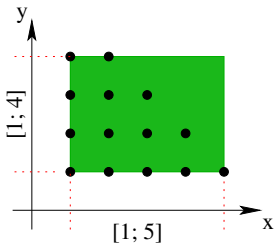
$\forall x \in D, \forall x^\# \in D^\#, x \in \gamma(x^\#) \Rightarrow f(x) \sqsubseteq \gamma(f^\#(x^\#))$

$$\begin{array}{ccc} \{1; 2; 3\} & \xrightarrow{f(x) = x^2} & \{1; 4; 9\} \sqsubseteq \{1; 2; 3; 4; 5; 6; 7; 8; 9\} \\ \cap & & \uparrow \\ [1; 3] & \xrightarrow{f^\#} & [1; 9] \text{ --- } \gamma \end{array}$$

Garantie de correction $\Rightarrow$

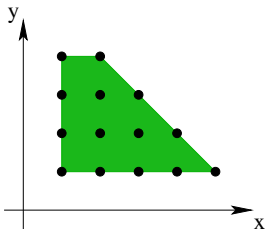
- « L'analyse n'oublie pas de valeurs ».
- « L'analyse n'oublie pas de bugs ».

- Approximation par des intervalles : peu précis.



- Ne tient pas compte des relations entre les variables.
- Représente beaucoup de traces non réelles.

- Gain de précision avec un domaine **relationnel**.
- Tient compte des relations entre les variables.



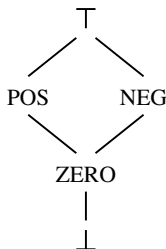
- Nombreux domaines abstraits relationnels
 - ▶ polyèdres, octogones, ellipses. . .
- Généralement plus complexes et plus coûteux.

Syntaxe du langage

$i ::= v := e \mid i; i$
| if $v > 0$ then i else i
| while $v > 0$ do i done

$e ::= v \mid n$
| $e + e \mid e - e \mid e * e \mid e / e$

Treillis



Remarques

- Langage minimaliste.
- Construction while transformée en **test** et **saut** dans le CFG.
- Test avec une **variable** (pas expression quelconque) :
 - Compilation $e : \rightarrow$ résultat dans 1 variable.
 - $\Rightarrow$ Pas de perte d'expressivité.

Abstraction

$$\alpha(S) = \begin{cases} \top & \text{si } \exists x, y \in S, x < 0 \wedge y > 0 \\ NEG & \text{si } \forall x \in S, x \leq 0 \wedge \exists y \in S, y < 0 \\ ZERO & \text{si } S = \{0\} \\ POS & \text{si } \forall x \in S, x \geq 0 \wedge \exists y \in S, y > 0 \\ \perp & \text{si } S = \emptyset \end{cases}$$

Concrétisation

$$\begin{aligned} \gamma(\top) &= \mathbb{Z} \\ \gamma(NEG) &=]-\infty; 0] \\ \gamma(ZERO) &= \{0\} \\ \gamma(POS) &= [0; +\infty[\\ \gamma(\perp) &= \emptyset \end{aligned}$$

Correspondance de Galois S'en convaincre en exercice.

Opérations abstraites

- $n^\# = \alpha(\{n\}) = \begin{cases} NEG & \text{si } n < 0 \\ ZERO & \text{si } n = 0 \\ POS & \text{si } n > 0 \end{cases}$

- $x^\# +^\# y^\# = \alpha(\{x + y \mid x \in \gamma(x^\#), y \in \gamma(y^\#)\}) =$

$+^\#$	$\top$	NEG	$ZERO$	POS	$\perp$
$\top$	$\top$	$\top$	$\top$	$\top$	$\perp$
NEG	$\top$	NEG	NEG	$\top$	$\perp$
$ZERO$	$\top$	NEG	$ZERO$	POS	$\perp$
POS	$\top$	$\top$	POS	POS	$\perp$
$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$

- $x^\# *^\# y^\# = \alpha(\{x * y \mid x \in \gamma(x^\#), y \in \gamma(y^\#)\}) =$

$*^\#$	$\top$	NEG	$ZERO$	POS	$\perp$
$\top$	$\top$	$\top$	$ZERO$	$\top$	$\perp$
NEG	$\top$	POS	$ZERO$	NEG	$\perp$
$ZERO$	$ZERO$	$ZERO$	$ZERO$	$ZERO$	$\perp$
POS	$\top$	NEG	$ZERO$	POS	$\perp$
$\perp$	$\perp$	$\perp$	$\perp$	$\perp$	$\perp$

- $x^\# -^\# y^\# = \dots, \text{etc.}$

Utilise un **état** ρ : variable $\mapsto$ valeur abstraite.

Expressions

- $\llbracket v \rrbracket_{\rho}^{\sharp} = \rho(v)$
- $\llbracket n \rrbracket_{\rho}^{\sharp} = n^{\sharp}$
- $\llbracket e1 + e2 \rrbracket_{\rho}^{\sharp} = \llbracket e1 \rrbracket_{\rho}^{\sharp} +^{\sharp} \llbracket e2 \rrbracket_{\rho}^{\sharp}$
- $\llbracket - \ * \ / \rrbracket_{\rho}^{\sharp} = \dots$

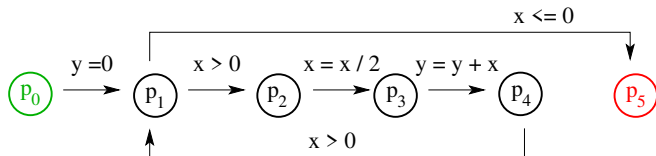
Instructions

- $\llbracket v := e \rrbracket_{\rho}^{\sharp} = \rho \oplus [v \mapsto \llbracket e \rrbracket_{\rho}^{\sharp}]$
- $\llbracket v > 0 \rrbracket_{\rho}^{\sharp} = \rho \oplus [v \mapsto \rho(v) \sqcap^{\sharp} POS]$

Comment calculer l'analyse

- Travailler sur le CFG du programme.
- Arc entre 2 points l et l' avec instruction $i : (l, i, l')$.
- Ensemble des arcs noté $\mathcal{A}$.
- Associer un état ρ_l à chaque point de programme l .
- Trouver la plus petite solution de
$$\begin{cases} \rho_0 = [\mathbb{V} \mapsto \perp] \\ \rho_{l'} = \bigsqcup_{(l, i, l') \in \mathcal{A}} \llbracket i \rrbracket_{\rho_l}^\# \end{cases}$$
- Méthode itérative :
 - ➊ Début : $\forall l \in \mathcal{A}, \rho_l^0 = [\mathbb{V} \mapsto \perp]$.
 - ➋ Calculer $\rho_{l'}^{k+1} = \bigsqcup_{(l, i, l') \in \mathcal{A}} \llbracket i \rrbracket_{\rho_l^k}^\#$.
 - ➌ Retourner en 2 jusqu'à atteindre un point fixe.
- Pour accélérer, on calcule $\rho_{l'}^{k+1}$ en fonction de ρ_l^{k+1} lorsque possible.

Exemple



$$\rho_0^{k+1} = [x \mapsto \perp; y \mapsto \perp]$$

$$\rho_1^{k+1} = (\rho_0^{k+1} \oplus [y \mapsto ZERO]) \sqcup \rho_4^k$$

$$\rho_2^{k+1} = \rho_1^{k+1} \oplus [x \mapsto \rho_1^{k+1}(x) \sqcap^\# POS]$$

$$\rho_3^{k+1} = \rho_2^{k+1} \oplus [x \mapsto \rho_2^{k+1}(x) /^\# 2^\#]$$

$$\rho_4^{k+1} = \rho_3^{k+1} \oplus [y \mapsto \rho_3^{k+1}(y) +^\# \rho_3^{k+1}(x)]$$

$$\rho_5^{k+1} = \rho_4^{k+1} \oplus [x \mapsto \rho_1^{k+1}(x) \sqcap^\# NEG]$$

- 1 Créer le jeu d'équations.
- 2 Calculer le point fixe de l'ensemble d'équations.

- Technique d'analyse **sans exécuter** le programme.
- Repose sur des **fondements mathématiques** complexes.
- Nécessite des **propriétés** sur les **domaines** et **fonctions** manipulés.
- $\Rightarrow$ Apporte la garantie de **correction**.
- Outils industriels (APRON, Astrée, CGS, Fluctuat, Frama-C, Polyspace. . .)
- Nombreux aspects non étudiés ici :
 - widening,
 - narrowing,
 - domaines relationnels. . .

Principes des lang. de progr.

INE 213

François Pessaux

ENSTA ParisTech, édition 2023-2024

prénom.nom@ensta.fr



Quelques subtilités de langages

Le code

```
#include <stdio.h>

int main (void) {
    signed char i = -128 ;
    i = (i - 1) / 2 ;
    printf ("%d\n", i) ;
    return (0) ;
}
```

Souvenez-vous

- char = 8 bits.
- Arithmétique entière : modulo.
- Entiers signés : représentation en complément à 2.

Qu'affiche ce programme ?

On pourrait penser que

- signed char = 7 bits + 1 bit de signe.
- -128 : plus petite valeur représentable ($0x80$).
- $-128 - 1 = 0x80 - 1 = 0xF \dots F7F \rightsquigarrow 0x7F = 127$.
- $127/2 = 63$.

Affichage

```
$ ./a.out  
-64
```

Explication

- Norme de C : calculs intermédiaires sur taille du **mot machine**.
- $\Rightarrow 32$ (ou 64) bits.
- $\Rightarrow$ Pas de débordement : $-128 - 1 = -129$, $-129/2 = -64$.

Le code

```
#include <stdio.h>

int main (void) {
    char t[2] = { 'A', 'B' } ;
    printf ("%c\n", t[1]) ;
    return (0) ;
}
```

Compile, compile pas, fait quoi ?

Affichage

```
$ gcc -Wall foo.c  
$ ./a.out  
B
```

Explication

- Tableau $\equiv$ **pointeur** sur son début.
- $t[i]$: adresse de $t + i * \text{taille donnée pointée}$.
- Arithmétique sur pointeur : $p + i = p + i * \text{taille donnée pointée}$.
- $t[i] \equiv t + i$.
- Addition **commutative**.
- $\Rightarrow$ On sait compiler : on accepte dans le langage.

Le code

```
#include <stdio.h>

int main (int argc, char *argv[]) {
    int a = atoi (argv[1]) ;
    printf ("%d/%d = %d\n", a, a, a / a) ;
    return (0) ;
}
```

Qu'affiche ce programme pour 0 ?

C – Division blindée (1/2)

Ça dépend...

- `$ gcc foo.c`
`$ ./a.out 0`
Floating point exception: 8
- `$ gcc foo.c -O1`
`$ ./a.out 0`
`0/0 = 1`

Optimisations optimisent parfois trop

- Analyse simple du code : $a/a \rightarrow 1$.
- Ajouter un test **juste** pour le cas $0 \leadsto$ code plus **long**.
- Donc pas « optimisé » par rapport à juste dire « ça vaut 1 »
- $\Rightarrow$ Choix **arbitraire** du compilateur : ignorer la $/0$.

```
idivl -20(%rbp)
movl -24(%rbp), %ecx
movl %ecx, %edx
movl %eax, %ecx
movb $0, %al
callq _printf
```

$\Rightarrow$

```
movl $1, %ecx
xorl %eax, %eax
movl %edx, %esi
callq _printf
```

Le code

```
#include <stdio.h>

int main () {
    int i ;
    char c ;

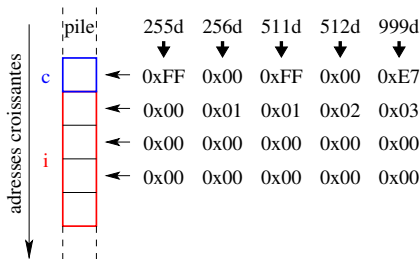
    for (i = 0; i < 3; i++) {
        scanf ("%d", &c) ;
        printf ("d: %d\n", i) ;
    }
    return (0) ;
}
```

Qu'affiche ce programme sur un processeur Intel ?

C – scanf grand format (2/2)

Affichage

```
$ ./a.out  
512  
d: 2  
$ ./a.out  
511  
d: 1  
20  
d: 0  
~C
```



Explication

- Mauvais **format** scanf ("%d", &c)
- **32** bits écrits (et non 8) à partir de l'adresse de c.
- **Écrasement** d'octets de i.
- Entrée < 256 $\Rightarrow$ octets débordant dans i $\leadsto$ i = 0.
- Entrée < 512 $\Rightarrow$ octets débordant dans i $\leadsto$ i = 1.
- $\Rightarrow$ Boucle sans fin si entrée < 512, sortie directe sinon.

Ruby – On ne devrait pas être affecté (1/2)

Le code

```
$ irb
irb(main):001:0> foo
NameError: undefined local variable or method 'foo' for main:Object
from (irb):1
from /usr/bin/irb:11:in '<main>'

irb(main):002:0> foo = bar
NameError: undefined local variable or method 'bar' for main:Object
from (irb):2
from /usr/bin/irb:11:in '<main>'

irb(main):003:0> foo = foo
```

Résultat de l'affectation `foo = foo` ?

Ruby – On ne devrait pas être affecté (2/2)

Le code

```
$ irb
irb(main):001:0> foo
NameError: undefined local variable or method 'foo' for main:Object
  from (irb):1
  from /usr/bin/irb:11:in '<main>'

irb(main):002:0> foo = bar
NameError: undefined local variable or method 'bar' for main:Object
  from (irb):2
  from /usr/bin/irb:11:in '<main>'

irb(main):003:0> foo = foo
=> nil
```

Rappel

- [] : tableau.
- {} : objet.

Le code

```
$ jsc
>>> [] + []

>>> {} + {}
NaN
>>> {} + []
0
>>> [] + {}
[object Object]
>>> 0 + {}
0[object Object]
>>> {} + 0
0
```

Le code

```
$ jsc
>>> if (0=='0') print ("Equal") ; else print ("Different") ;
Equal
>>> switch (0)
... { case '0' : print ("Equal") ; break ;
...   default : print ("Different") ; break ; }
Different
```

(print () ou console.log ())

JavaScript – En surcharge (1/2)

Le code

```
$ jsc
>>> if ('0' == 0) print("'0' == 0") ; else print("'0' <> 0") ;
'0' == 0
>>> if (0 == '0.0') print("0 == '0.0'") ; else print("0 <> '0.0'") ;
0 == '0.0'
>>> if ('0' == '0.0') print("'0' == '0.0'") ; else print("'0' <> '0.0'") ;
```

Résultat de '0' == '0.0' ?

Le code

```
$ jsc
>>> if ('0' == 0) print("'0' == 0") ; else print("'0' <> 0") ;
'0' == 0
>>> if (0 == '0.0') print("0 == '0.0'") ; else print("0 <> '0.0'") ;
0 == '0.0'
>>> if ('0' == '0.0') print("'0' == '0.0'") ; else print("'0' <> '0.0'") ;
'0' <> '0.0'
```

- Surcharge de l'opérateur ==.
- S'adapte aux **types des opérandes**.
- N'est donc plus **transitif** si utilisé avec des **types différents**.

Le code

```
$ python3
>>> row = [0, 0, 0]
>>> matrix = [row, row, row]
>>> row[1] = 2
>>> matrix
```

Que vaut matrix?

Le code

```
$ python3
>>> row = [0, 0, 0]
>>> matrix = [row, row, row]
>>> row[1] = 2
>>> matrix
```

Réponse : `[[0, 2, 0], [0, 2, 0], [0, 2, 0]]`

- Python favorise le partage via des **pointeurs**.
- `matrix` reçoit 3 fois la **même** adresse du tableau `row`.

Même certains entiers

`x is y` : teste si `x` et `y` pointent la même adresse.

```
$ python3
>>> x = 5
>>> y = 5
>>> x is y
True
>>> z = 500
>>> t = 500
>>> z is t
False
```

Le code

```
$ python3
>>> a = b = [0]
>>> a += b
>>> a, b
([0, 0], [0, 0])
```

Et maintenant

```
$ python3
>>> a = b = [0]
>>> a = a + b
>>> a, b
([0, 0], [0])
```

Si, si, c'est vraiment nul

```
$ jsc  
>>> new Array() == 0  
true  
>>> new Array() == false  
true
```

Le code

```
(* blah blah " blah blah *)  
let x=true;;  
(* PREVIOUS VERSION -----  
(* blah blah " blah blah *)  
let x=false;;  
(* blah blah " blah blah *)  
-----*)  
(* blah blah " blah blah *)
```

Que fait ce code ?

Le code

```
(* blah blah " blah blah *)  
let x=true;;  
(* PREVIOUS VERSION -----  
(* blah blah " blah blah *)  
let x=false;;  
(* blah blah " blah blah *)  
-----*)  
(* blah blah " blah blah *)
```

Réponse : définit une liaison `x = false`.

- Possibilité d'ouvrir une chaîne dans un commentaire.
- $\Rightarrow$ Risque **d'induire en erreur** lors de **relecture**.

Le code

```
#include <stdio.h>

int foo () {
    int a = 4 ;
    int b = 2 ;
    return a /**
                /*/ b
;
}

int main (void) { printf ("%d\n", foo ()) ; return 0 ; }
```

Que fait ce fait ce programme ?

Le code

```
#include <stdio.h>

int foo () {
    int a = 4 ;
    int b = 2 ;
    return a /*
                */ b
;
}
```

Ça dépend de la compilation

- GCC sans option particulière : affiche 4.
- GCC avec option `-std=c89` : affiche 2.
- C89, pas de commentaire « uniligne » : tout entre `/*` et `*/` supprimé.
- C « moderne » : tout derrière `//` supprimé.

Importantes différences entre Python 2.x et 3.x

Python 2.7.15

```
>>> 1 / 2
0
>>> print 3
3
>>> print (3, 3)
(3, 3)
>>> l = [s+1 for s in [1,2,3]]
>>> s
3
>>> round (6.5)
7.0
```

Python 3.6.8

```
>>> 1 / 2
0.5
>>> print 3
SyntaxError: Missing parentheses
>>> print (3, 3)
3 3
>>> l = [s+1 for s in [1,2,3]]
>>> s
NameError: name 's' is not defined
>>> round (6.5)
6
```

Conséquences

- Relecture impossible sans la version.
- Portabilité médiocre entre versions.
- Risque d'induire en erreur les non-spécialistes.

Le code

```
#include <stdio.h>
int main () {
    int s = - 1 ;
    unsigned u = 1 ;

    if (s < u) printf ("-1 < 1\n") ;
    else printf ("1 < -1\n") ;
    return (0) ;
}
```

Qu'affiche ce programme ?

Le code

```
#include <stdio.h>
int main () {
    int s = -1 ;
    unsigned u = 1 ;

    if (s < u) printf ("-1 < 1\n") ;
    else printf ("1 < -1\n") ;
    return (0) ;
}
```

Réponse : 1 < -1

- Mélange entre **signé** et **non signé**.
- Norme de C : le signé est **converti** en non signé.
- $\Rightarrow$ Bit de signe devient partie de la **magnitude**.
- Bit de poids fort = 1 $\Rightarrow$ valeur très > 1 !

Indication

$12345678934324999 / 2 = 6172839467162499$

Le code

```
$ python3
>>> def f (x) :
...     x = x / 2
...     return (x % 2) == 0
...
>>> f (12345678934324999)
```

Quelle valeur retournée en Python3(.10.11) ?

Le code

```
$ python3
>>> def f (x) :
...     x = x / 2
...     return (x % 2) == 0
...
>>> f (12345678934324999)
```

Réponse : True

Explication

- Calculs en flottant par défaut.
- Flottants pas tous représentables : arrondis (trous).
- 12345678934324999 arrondi à 12345678934325000.
- Divisé par 2 $\rightarrow$ 6172839467162500 $\Rightarrow$ bien pair.
- $\Rightarrow$ Utiliser explicitement `//`.

OCaml– Mutation sans gêne (1/2)

Avertissement : avant la version 4.02.

Le code

```
# let check c = if c then "Tout va bien" else "Tout va mal!" ;;  
val check : bool → string = <fun>  
# let f = check false in  
  f.[8] ←  $\beta$ '; f.[9] ← 'i'; f.[10] ← 'e'; f.[11] ← 'n' ;;  
- : unit = ()
```

Valeur de check true ? De check false ?

Réponse :

```
check true ;;  
- : string = "Tout va bien"  
# check false ;;  
- : string = "Tout va bien"
```

- Chaînes statiques étaient **mutables**.
- $\Rightarrow$ Effet de bord : modification **permanente** et **partagée**.
- Depuis version 4.02 différenciation :
 - chaînes mutables : `Bytes`,
 - chaînes immutables : `String`.

Le code

```
#include <string.h>

class Str {
    int strlen (int l) {
        return strlen ("Foobar") ;
    }
};
```

Comment se comporte ce programme ?

C++- Surcharge, ou pas (2/2)

Le code

```
#include <string.h>

class Str {
    int strlen (int l) {
        return strlen ("Foobar") ;
    }
};
```

Compilation

```
$ g++ foo.cpp
foo.cpp:9:20: error: cannot initialize a parameter of type 'int' with an
      lvalue of type 'const char [7]'
      return strlen ("Foobar") ;
                  ~~~~~~
foo.cpp:8:19: note: passing argument to parameter 'l' here
      int strlen (int l) {
                  ^
1 error generated.
```

Pas de surcharge :

- Pas de méthode strlen : string → int dans la classe.
- strlen de string.h ≠ classe Str.
- ⇒ strlen considérée récursive ⇒ mal typé.

OCaml – L'optionnel n'est pas une option

Le code

```
# let set_pin ?secret:(v=0) user =  
  (* Do something really interesting . *)  
  Printf.printf "PIN registered for %s.\n" user ;;  
val set_pin : ?secret:int → string → unit = <fun>  
# set_pin ~secret: 1234 "bob" ;;  
PIN registered for bob.  
- : unit = ()
```

Le danger

```
# set_pin "root" ;;  
PIN registered for root.  
- : unit = ()
```

- L'oubli de l'argument passe inaperçu.
- Était-ce un oubli ou volontaire ?
- $\Rightarrow$ Jette un doute lors d'une relecture.
- Se méfier des constructions de « confort » / de « paresse ».
- Certaines constructions sont potentiellement dangereuses.

C – Ça doit rentrer dans les cases (1/2)

Le code

```
#include <stdio.h>
int main (void) {
    int j = 24 ;

    switch (j) {
    case 23 : {
        printf ("cas 23\n\n\n") ;
        while (j) {
            default :
                printf("cas-du-default \n") ;
        }
    }
    break ;
}
printf ("\n\n\n") ;
return (0) ;
}
```

Compile, compile pas, fait quoi ?

C – Ça doit rentrer dans les cases (2/2)

Le code

```
#include <stdio.h>
int main (void) {
    int j = 24 ;

    switch (j) {
    case 23 : {
        printf ("cas 23\n\n\n") ;
        while (j) {
            default :
                printf("cas-du-default \n") ;
        }
    }
    break ;
}
printf ("\n\n\n") ;
return (0) ;
}
```

Réponse

```
$ gcc -Wall foo.c
$ ./a.out
cas-du-default
cas-du-default
cas-du-default
cas-du-default
^C
```

- Construction switch : pas de structure particulière.
- Syntaxe :
$$instr ::= \textbf{switch} \ expr \ instr \mid \textit{case} : \ instr \mid \dots$$
$$\textit{case} ::= \textbf{case} \ cst \mid \textbf{default}$$
- Étiquettes pouvant apparaître n'importe où.
- Compilé comme des sauts aux étiquettes.

C – Il faut que le proto colle (1/3)

Fichier bar.c

```
#include <stdlib.h>

double* f (int n) {
    return (malloc (n * sizeof (double))) ;
}
```

Fichier foo.c

```
#include <stdio.h>

int main (void) {
    double *t = f (2) ;
    for (int i = 0; i < 2; i++) {
        t[i] = i ;
        printf ("%lf\n", t[i]) ;
    }
    return (0) ;
}
```

Qu'affiche ce programme ?

C – Il faut que le proto colle (2/3)

Fichier

bar.c

```
#include <stdlib.h>

double* f (int n) {
    return (malloc (n * sizeof (double))) ;
}
```

Fichier

foo.c

```
#include <stdio.h>

int main (void) {
    double *t = f (2) ;
    for (int i = 0; i < 2; i++) {
        t[i] = i ;
        printf ("%lf\n", t[i]) ;
    }
    return (0) ;
}
```

Réponse :

```
$ gcc foo.c bar.c
foo.c:6:15: warning: implicit declaration of function 'f' is invalid in C99
             [-Wimplicit-function-declaration]
    double *t = f (2) ;
                  ^
foo.c:6:11: warning: incompatible integer to pointer conversion initializing
             'double *' with an expression of type 'int' [-Wint-conversion]
    double *t = f (2) ;
                  ^~~~~~
2 warnings generated.
$ ./a.out
Segmentation fault: 11
```


Explication

- Pas de **prototype** de `f` connu dans `main`.
 - Norme de C : suppose retourner **`int`**.
 - `sizeof (int) = 4`, `sizeof (double*) = 8`.
 - `main` récupère **4** octets ... et l'étend sur **8**.
- t in f: 0x7f95984006a0
t in main: 0xffffffff984006a0
- Extension **signée** (puisque `int` supposé) $\Rightarrow$ adresse **incorrecte**.

```
...  
callq _f  
leaq L_.str(%rip), %rdi  
movslq %eax, %rcx ; <--- Move sign-extended double word  
...
```

Rappels

- $100000 \times 100000 = 10000000000 = 10^{10}$.
- `long int` = 64 bits (dont 1 de signe) et $10^{10} < 2^{63}$.

Le code

```
#include <stdio.h>

int main(void) {
    int a = 100000, b = 100000 ;
    long int c = a * b ;
    printf ("%ld\n", c) ;
    return (0) ;
}
```

Et alors ?

Réponse :

```
$ gcc -Wall foo.c  
$ ./a.out  
1410065408
```

Forcer la taille du résultat

```
...  
    c = (long int) (a * b) ;  
...
```

⇒ Même résultat.

Explication et solution

- Norme de C : 2 opérandes **int** ⇒ calcul sur int.
- ⇒ Débordement et troncature **avant** extension et affectation dans le long int.
- Solution : forcer **taille d'opérande** :
 - ▶ `c = (long int) a * b ;`

- Choix de conception du langage : influence sa **sémantique**.
- Parfois, sémantique **compliquée** :
 - Source de bugs.
 - Relecture difficile.
- Parfois sémantique **arbitraire** $\Rightarrow$ pas forcément intuitif.
- Se méfier des constructions « magiques » :
 - ▶ l'**explicite** vaut mieux que l'implicite.

Court exposé sur la question, avec beaucoup d'humour ☺

<https://www.destroyallsoftware.com/talks/wat>